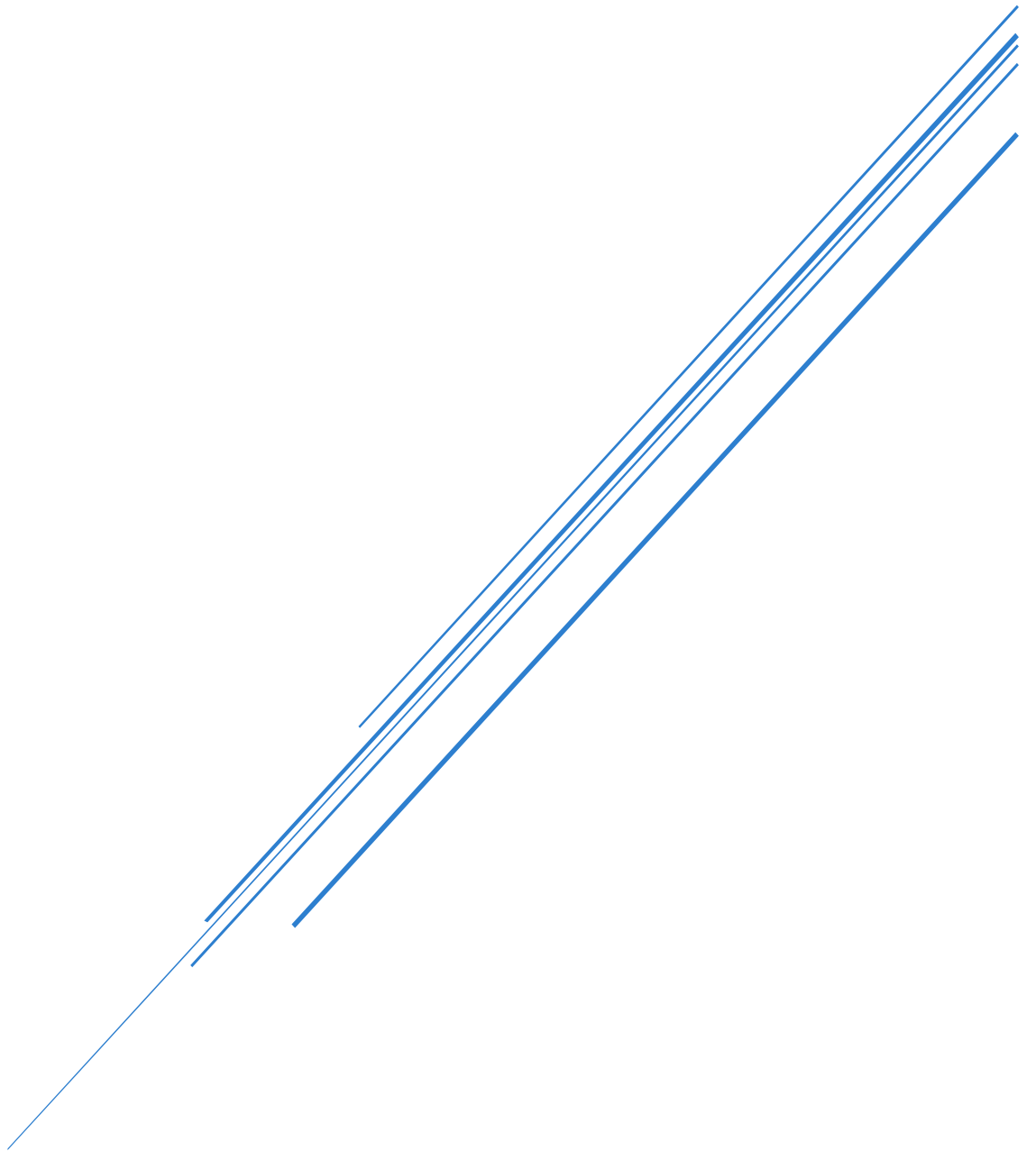


PROJECTE INTERMODULAR ASIX HOSPITAL

Hospital Tenso



Eric Lopez i Nil Parra
19/05/2026

Índex

Introducció al projecte	3
Disseny de base de dades	4
Model entitat relació	4
Esquema de seguretat base de dades	5
Explicació general	5
Esquema de seguretat	6
Configuració SSL	7
Data masking	9
Document AGPD	10
Connectivitat i login	11
Connexió amb la base de dades	11
Inici de sessió	12
Registre d'usuaris	12
Gestió de sessió i tancament	13
Frontend	13
Bloc de manteniment	14
Explicació general	14
Esquema d'alta disponibilitat	14
Proposta maquinari	14
Proposta model de replicació	15
Esquema de infraestructura	15
Descripció del disseny	16
Resum workflow	16
Implementació	16
Proposta model de backups	28
Pla de còpies de seguretat	28
Pla d'actuació	29
Bloc de consultes	30
Explicació general	30
Dummy Data	30
Disseny general de la solució	30
Generació de les dades	31
Consistència i qualitat del Dummy data	33
Índexs i rendiment	34

Integració dins l'aplicació	35
Validació prèvia i mesures de seguretat	37
Eliminació del Dummy data	37
Bloc d'exportació i validació de dades	38
Objectiu d'aquesta funcionalitat	38
Funcionament general	38
Dades que conté l'exportació	39
Exportació en format JSON	39
Exportació en format XML	40
Validació amb JSON Schema i XSD	41
Dashboard PowerBI	41
Objectiu del Dashboard	41
Font de dades i connexió	41
Desglossament de visites per àrea	42
Altres visuals de suport	43
Disseny i resultat final	43
Enllaços d'interès	44
Annex 1	45
Model relacional	45
Script SQL implementació	49
Annex2	58
Esquema de seguretat	58
Annex 3	68
Instal·lació tailscale	68
Motiu d'ús	68
Servidor Local	68
Servidor Cloud	70
Annex 4	73
Esquema json	73
Esquema XSD	75

Introducció al projecte

L'objectiu d'aquest projecte es posar en practica les habilitats i coneixements adquirits durant el primer curs d'ASIX en els mòduls de: gestió de bases de dades, administració de sistemes gestors de bases de dades, introducció a la programació i llenguatge de marques i gestió d'informació.

En aquest projecte se'ns demana idear un sistema informàtic per la gestió d'un hospital des de la manera en que es gestionen les dades, el programa de gestió i diferents mesures de seguretat.

La nostra solució consta de les següents tecnologies:

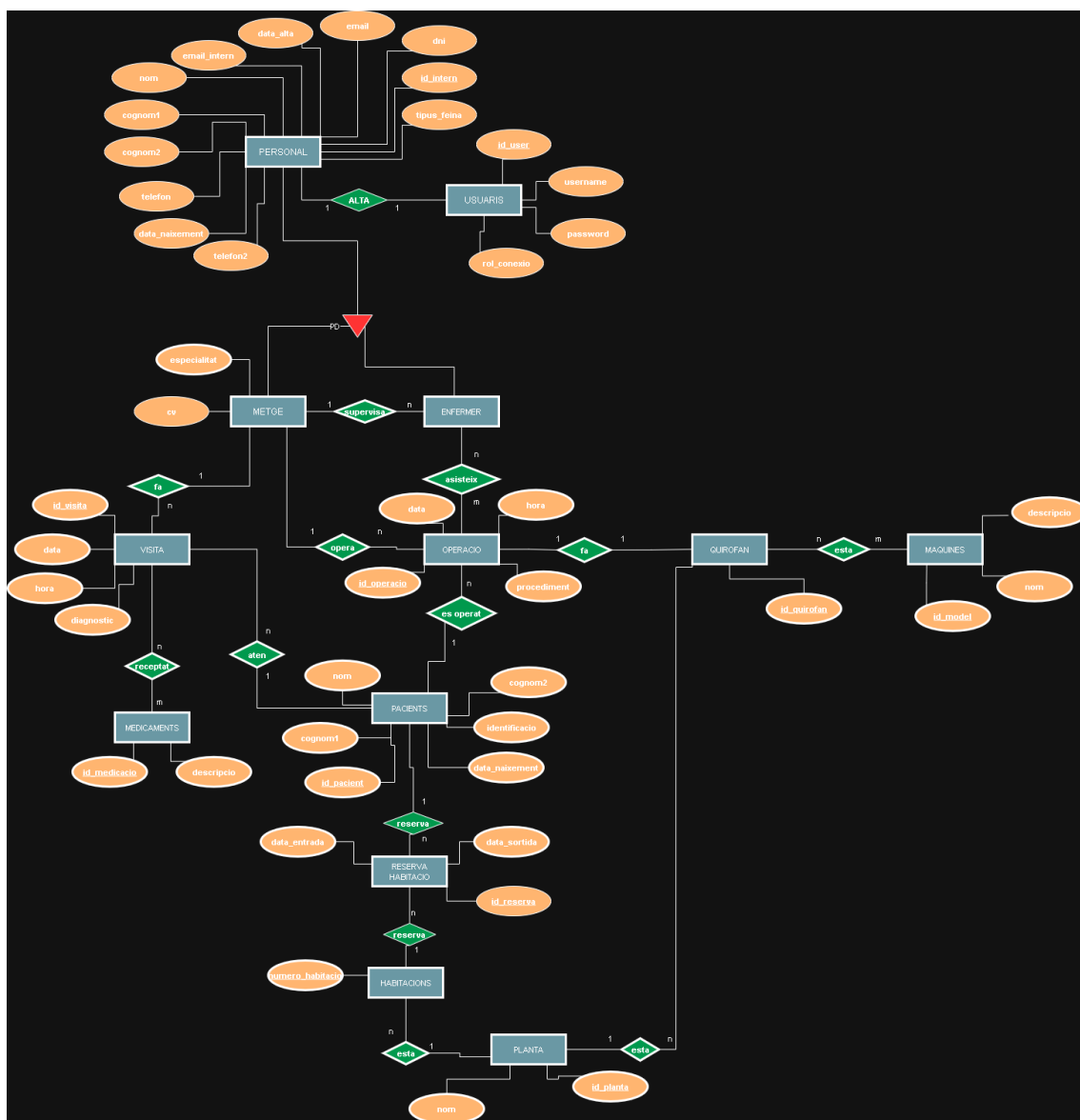
- Frontend:** tkinter(Python)
- Backend:** flask(Python)
- Base de dades:** Postgresql
- VPN:** tailscale
- Servidor de replica:** digital ocean

Disseny de base de dades

En aquest apartat es mostrarà el disseny de la base de dades. Per informació més completa anar a [l'annex 1](#).

Model entitat relació

El model relacional resultant té en compte totes les peticions recollides i està enfocat en un millor rendiment per la base de dades, aquest esquema ha patit diferents modificacions degut a necessitats de programa.



Esquema de seguretat base de dades

Explicació general

A la base de dades s'ha definit una matriu de seguretat per poder separar els diferents tipus d'usuaris i controlar l'accés d'una forma més segura. S'han creat 2 usuaris tècnics principals hosp_admin que té tots els permisos d'administració i hosp_app que serà l'usuari que utilitza el backend de l'aplicació. A més hem creat diferents tipus de rols depenent de l'usuari (administrador, personal, sanitari, gestio, serveis i pacient) i també rols de login (rol_metge, rol_infermer, rol_administratiu, rol_tecnic, rol_personal_neteja, rol_personal_seguretat, rol_personal_cuina, rol_pacient, rol_administrador).

Els permisos sobre les taules s'han distribuït segons el perfil. Per exemple el rol administratiu té accés a tot, el personal sanitari només a les dades assistencials, el personal de gestió accedeix només a les taules de suport necessàries, etc...

La part funcional dels usuaris es desa a la taula usuaris, on es guarda el nom d'usuari, la contrasenya encriptada, el rol i la relació amb el personal intern. Quan un usuari inicia sessió el backend comprova la contrasenya i assigna el rol depenent l'usuari que sigui. Això permet separar la identitat de cada usuari i el seu rol.

Per el SSL hem utilitzat openssl per generar els certificats auto firmats tant per l'aplicació web com per el servidor on esta la base de dades. Això es fa per garantir que totes les dades viatgen per internet xifrades i garantint la seguretat de l'aplicació i de les dades.

Pel que fa el data masking hem implementat una solució en el backend que comprova el rol de l'usuari que es connecta i mostra totes les dades o las emmascara depenent del rol i l'usuari. D'aquesta manera la informació sensible només es mostra clar als rols amb Access complet.

Esquema de seguretat

Per complir amb els requisits demanats hem creat un fitxer .SQL on creem tots els rols i donem els permisos necessaris a cada objecte de la base de dades. Aquest fitxer ens permet desplegar tota la infraestructura de permisos de manera ràpida i eficient en cas que hi hagi algun inconvenient. Mes informació a [l'annex2](#)

Configuració SSL

En aquest apartat hem utilitzat openssl per crear un certificat auto firmat tant per l'aplicació web com per la connexió amb el servidor on esta la base de dades postgres, d'aquesta manera la connexió tant a l'aplicació com al servidor van xifrades i evitem atacs o accessos no autoritzats.

Aquí podem veure com s'ha creat el certificat en el servidor:

[illegible]

En el codi tenim que afegir les variables d'entorn perquè trobi el certificat i la clau, necessàries per poder establir la connexió SSL, també canviar que obri el port 443 i afegir el `ssl_context`.

```
DB_HOST = localhost
DB_DATABASE = hosp_blanes
DB_USER = hosp_app
DB_PASSWORD = P@ssw0rd!app

DB_PORT = 5432
FLASK_SECRET = ZXJpY2xvcGV6bm1scGFfycmE
UPLOAD_FOLDER = \\uploads
ALLOWED_EXTENSIONS = {'pdf'}

CERT=/etc/certificates/flask/cert.pem
KEY=/etc/certificates/flask/key.pem
```


Per el postgres:

Tenim que modificar una sèrie de línies en el arxiu postgresql.conf, activem el ssl i li posem les rutes a on està el certificat i la clau per que les pugui trobar i crear la connexió:

```
# - SSL -

ssl = on
#ssl_ca_file = ''
ssl_cert_file = '/etc/certificates/postgres/server.crt'
#ssl_crl_file = ''
#ssl_crl_dir = ''
ssl_key_file = '/etc/certificates/postgres/server.key'
#ssl_ciphers = 'HIGH:MEDIUM:+3DES:!aNULL' # allowed SSL ciphers
#ssl_prefer_server_ciphers = on
#ssl_ecdh_curve = 'prime256v1'
#ssl_min_protocol_version = 'TLSv1.2'
#ssl_max_protocol_version = ''
#ssl_dh_params_file = ''
#ssl_passphrase_command = ''
#ssl_passphrase_command_supports_reload = off
```

Després al pg_hba.conf afegim que utilitzi scram-sha-256 per fer la connexió:

```
# Database administrative login by Unix domain socket
local all postgres peer

# TYPE DATABASE USER ADDRESS METHOD
# "local" is for Unix domain socket connections only
local all all peer
# IPv4 local connections:
hostssl all all 127.0.0.1/32 scram-sha-256
# IPv6 local connections:
#host all all ::1/128 scram-sha-256
# Allow replication connections from localhost, by a user with the
# replication privilege.
local replication all peer
host replication all 127.0.0.1/32 scram-sha-256
host replication all ::1/128 scram-sha-256
```

Per últim al codi posem que per fer la connexió a la base de dades requereixi la connexió ssl.

```
con = psycopg2.connect(
    host=db_host,
    database=db_database,
    user=db_user,
    password=db_password,
    sslmode="require"
)
cursor = con.cursor()
```

Data masking

Per protegir les dades personals de grau alt, hem implementat una capa d'emascarament al backend. Els camps que hem considerat sensibles son: dni, telefon, telefon2, email, email_intern, data_naixement. També hem definit els rols que tindran accés complet a totes les dades que son: hosp_admin, administrador, rol_administrador. Si un usuari no té permisos complets aquests camps no es mostren totalment abans de que la API retorni les dades.

L'emascarament es fa a nivell d'aplicació i no modifica les dades originals de la base de dades, això permet conservar la informació real intacta i evitar problemes.

Definició de dades personals de grau alt i rols amb accés complet:

```
#####  
# ROLS AMB ACCÉS COMPLET  
#####  
FULL_ACCESS_ROLES = {  
    'hosp_admin',  
    'administrador',  
    'rol_administrador',  
}
```

```
#####  
# CAMPS SENSIBLES  
#####  
SENSITIVE_FIELDS = (  
    'dni',  
    'telefon',  
    'telefon2',  
    'email',  
    'email_intern',  
    'data_naixement',  
)
```

Hem creat un fitxer anomenat [masking.py](#) on hem posat totes les funcions necessàries per poder emascarar les dades que considerem de grau alt. Després en el arxiu principal afegim 2 funcions que s'encarreguen de comprovar el rol de cada usuari i apliquen el emascarament depenent de l'usuari. En la primera funció s'encarrega de gestionar les peticions API abans de enviar-la al client, si la consulta es correcta l'envia a la segona funció i aquesta és la que s'encarrega de emascarar les dades o no depenent el rol que sigui l'usuari.

```
def api_response(result, data_key='data'):  
    ok, payload = result  
  
    if ok:  
        payload = _mask_payload(payload)  
        return jsonify({'ok': True, data_key: payload})  
  
    error_text = str(payload)  
    status_code = 403 if 'permis' in error_text.lower() or 'permission' in error_text.lower()  
    return jsonify({'ok': False, 'error': error_text}), status_code
```

```

def _mask_payload(payload):
    role = session.get('role')

    if masking.can_view_full_data(role):
        return payload

    if isinstance(payload, dict):
        masked_payload = {}
        for key, value in payload.items():
            if isinstance(value, dict) or isinstance(value, list):
                masked_payload[key] = _mask_payload(value)
            else:
                masked_payload[key] = masking.mask_value(key, value)
        return masked_payload

    if isinstance(payload, list):
        return [_mask_payload(item) for item in payload]

    return payload

```

Document AGPD

A més de la implementació tècnica s'ha preparat el document de seguretat per una possible auditoria de l'AGPD. En aquest document s'identifiquen els tipus de dades que estem tractant, el nivell de sensibilitat de cada una i les mesures de seguretat aplicades.

Aquest document complementa el esquema de seguretat de la base de dades, ja que descriu quines dades es tracten i quines mesures de seguretat s'apliquen.

Enllaç al document AGPD: [Document de Seguretat \(AGPD\)](#)

Connectivitat i login

Connexió amb la base de dades

La connexió amb la base de dades es fa al backend, el sistema carrega les dades de connexió des-de un fitxer de l'entorn d'aquesta manera les credencials de la base de dades no queden escrites en el codi font de l'aplicació. La funció obre la base de dades amb psycopg2 i retorna la connexió i el cursor necessaris per executar les consultes necessàries. A més hi ha una funció que carrega el script SQL per inicialitzar la base de dades per tal de crear o reiniciar la base de dades.

```
#####
# CONNECTION TO DB
#####
def connect():
    load_dotenv()
    db_host = os.getenv("DB_HOST")
    db_database = os.getenv("DB_DATABASE")
    db_user = os.getenv("DB_USER")
    db_password = os.getenv("DB_PASSWORD")
    con = psycopg2.connect(
        host=db_host,
        database=db_database,
        user=db_user,
        password=db_password
    )
    cursor = con.cursor()
    return con, cursor
```

```
##### lleeerrriiccc, 4 days ago • Format code ...
# INIT DB
#####
def init_db():
    con, cur = connect()
    with open("base_de_dades/implementacio.sql", "r") as f:
        sql = f.read()
        cur.execute(sql)
    con.commit()
    cur.close()
    con.close()
```

Inici de sessió

L'inici de sessió es fa des de l'endpoint /login. Quan l'usuari envia el formulari, el backend comprova les credencials cridant la funció de login del mòdul de gestió. Aquesta funció busca el nom d'usuari a la taula usuaris, recupera la contrasenya guardada i la compara amb la que ha introduït l'usuari mitjançant bcrypt. Si les credencials son correctes es crea una sessió `session['username'] = username` i l'usuari és redirigit a la pàgina principal (home). Si les credencials no son correctes es torna a carregar el login amb un missatge de error.

```
#####
# LOGIN ROUTE
#####
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'GET':
        return render_template('login.html')

    username = request.form.get('username')
    password = request.form.get('password')

    ok, error = m.login(username, password)
    if ok:
        session['username'] = username
        return redirect(url_for('home'))
    return render_template('login.html', error=error)
```

Registre d'usuaris

El registre de nous usuaris es fa des del endpoint /register. El formulari demana el nom d'usuari, la contrasenya, la confirmació de la contrasenya i l'identificador intern del personal. Abans de crear el compte, el backend valida que no hi hagi camps buits, que les contrasenyes coincideixin i que l'identificador intern sigui numèric. Després, es comprova que el personal existeixi a la base de dades i que el nom d'usuari no estigui repetit. Si tot és correcte, la contrasenya es xifra amb bcrypt i es desa a la taula usuaris. Aquest registre està pensat per crear comptes vinculats a personal existent, no per crear pacients.

```
#####
# REGISTER ROUTE
#####
@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'GET':
        return render_template('register.html', error=None)

    username = request.form.get('username', '').strip()
    password = request.form.get('password', '')
    confirm_password = request.form.get('confirm_password', '')
    id_intern_raw = request.form.get('id_intern', '').strip()

    if not username or not password or not confirm_password or not id_intern_raw:
        return render_template('register.html', error='Completa todos los campos.')

    if password != confirm_password:
        return render_template('register.html', error='Las contraseñas no coinciden.')

    if not id_intern_raw.isdigit():
        return render_template('register.html', error='El id_intern debe ser un numero entero.')

    ok, error = m.register(username, password, int(id_intern_raw))
    if ok:
        return redirect(url_for('login'))

    return render_template('register.html', error=error)
```

Gestió de sessió i tancament

La seguretat del bloc d'autenticació es basa en la sessió Flask. Les futures pantalles principals només són accessibles si existeix `session['username'] = username`. També hi ha una ruta de tancament de sessió i retorna l'usuari a la pantalla de login. Això evita que es pugui continuar accedint a l'aplicació un cop finalitzada la sessió.

```
#####  
# LOGOUT ROUTE  
#####  
@app.route('/logout')  
def logout():  
    session.pop('username', None)  
    return redirect(url_for('login'))
```

Frontend

El frontend de l'aplicació s'ha desenvolupat amb HTML i CSS, mantenint una estructura simple i clara per facilitar la navegació de l'usuari final. En aquest apartat només es mostrarà el panell de login i registre perquè és lo que demana el bloc, més endavant es mostrarà tota la aplicació per dintre.

The image displays two side-by-side screenshots of a web application's frontend, specifically the login and registration sections.

Left Screenshot: Hospital Portal Login

- Title:** Hospital Portal
- Subtitle:** Sign in to access patient and staff systems.
- Form Fields:**
 - Username:** A text input field with the placeholder text "Enter your username".
 - Password:** A text input field with the placeholder text "Enter your password".
- Buttons:**
 - Sign In:** A prominent blue button.
 - Create Account:** A white button with a blue border.

Right Screenshot: Create Account

- Title:** Create Account
- Subtitle:** Register a new hospital user profile.
- Form Fields:**
 - Username:** A text input field with the placeholder text "Enter your username".
 - id_intern:** A text input field with the placeholder text "Enter personal id".
 - Password:** A text input field with the placeholder text "Create a password".
 - Confirm password:** A text input field with the placeholder text "Repeat your password".
- Buttons:**
 - Create Account:** A prominent blue button.
 - Back to login:** A white button with a blue border.

Bloc de manteniment

Explicació general

Aquest bloc consta d'aplicacions que ens permetin mantenir la aplicació, com per exemple poder afegir pacients, nous usuaris, personal, consultar les assignacions d'infermers, disponibilitat de professionals... El codi per aquest bloc es troba al següent [GITHUB](#)

Esquema d'alta disponibilitat

Proposta maquinari

Utilitzarem una arquitectura híbrida, un servidor local en l'hospital i l'altre en el cloud.

Principal:

CPU: 8 vCPU

RAM: 32 GB de RAM

Emmagatzemant: 1TB SSD NVMe amb RAID 1

Sistema Operatiu: Linux (Ubuntu Server 24.04.4 LTS)

Funcions:

Aplicació (backend)

Base de dades PostgreSQL PRIMARY

Gestió d'usuaris i operacions crítiques.

Backups, guardar les darrers 5 còpies al disc local i una còpia al cloud.

Guardar logs d'accés a la base de dades en un arxiu logs

Secundari:

CPU: 6 vCPU

RAM: 16 GB de RAM

Emmagatzemant: 1TB SSD NVMe

Sistema Operatiu: Linux (Ubuntu Server 24.04.4 LTS)

Funcions:

Les mateixes que el principal a nivell d'aplicació. Així a nivell d'aplicació garantim una estructura de 2 nodes ACTIU-ACTIU.

Replica de la base de dades PRIMARY. Si cau la principal salta a aquesta. Aquí utilitzariem una estructura de 2 nodes ACTIU-PASSIU.

Tecnologies i stack:

Base de dades: PostgreSQL 18.3

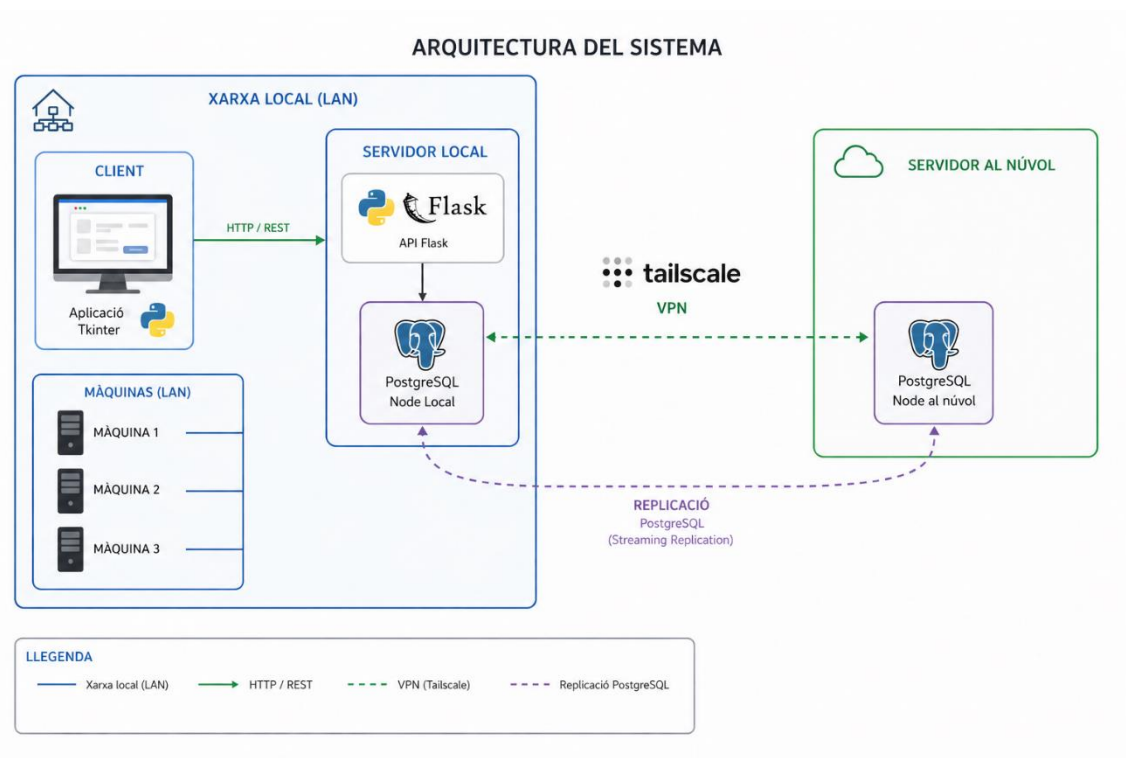
Aplicació (backend): Python, FLASK, Tkinter

Dashboard: PowerBI

VPN: Tailscale

Proposta model de replicació

Esquema de infraestructura



Descripció del disseny

La estructura de replicació es compon de dos servidors.

- Servidor local → node mestre
- Servidor cloud → node slave

La rèplica es farà utilitzant el mètode WAL(write ahead logging)

Resum workflow

- s'executa una transacció
- Es guarda la transacció al log de postgres
- Log de postgres massa gran i passa als arxius archive
- Carpeta archive es una carpeta compartida en xarxa amb NFS
- Servidor principal cau
- S'executa script
- S'aixeca servidor slave

Implementació

Servidor local

Configuració de postgres

El primer que farem com a bona pràctica es copiar el directori de configuració i el directori de postgres.

```
eric@serverhosp:~$ pwd
/home/eric
eric@serverhosp:~$ tree -f
.
├── ./postgres-backup
│   ├── ./postgres-backup/16
│   │   ├── ./postgres-backup/16/main
│   │   │   ├── ./postgres-backup/16/main/conf.d
│   │   │   ├── ./postgres-backup/16/main/environment
│   │   │   ├── ./postgres-backup/16/main/pg_ctl.conf
│   │   │   ├── ./postgres-backup/16/main/pg_hba.conf
│   │   │   ├── ./postgres-backup/16/main/pg_ident.conf
│   │   │   ├── ./postgres-backup/16/main/postgresql.conf
│   │   │   └── ./postgres-backup/16/main/start.conf
│   └── ./postgres-lib
│       ├── ./postgres-lib/16
│       │   ├── ./postgres-lib/16/main
│       └── ./postgres-lib/postgresql
│           ├── ./postgres-lib/postgresql/16
│           └── ./postgres-lib/postgresql/16/main [error opening dir]
11 directories, 6 files
eric@serverhosp:~$
```

Un cop fet apaguem el servei de postgres.

```
eric@serverhosp:~$ sudo !!
sudo systemctl status postgresql
● postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: enabled)
   Active: inactive (dead) since Wed 2026-04-29 15:54:29 UTC; 2min 41s ago
     Duration: 2h 15min 4.893s
    Main PID: 906 (code=exited, status=0/SUCCESS)
       CPU: 2ms

Apr 29 13:39:25 serverhosp systemd[1]: Starting postgresql.service - PostgreSQL RDBMS...
Apr 29 13:39:25 serverhosp systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
Apr 29 15:54:29 serverhosp systemd[1]: postgresql.service: Deactivated successfully.
Apr 29 15:54:29 serverhosp systemd[1]: Stopped postgresql.service - PostgreSQL RDBMS.
eric@serverhosp:~$ |
```

Ara començarem a editar la configuració de postgres.

Les línies que hem de canviar son les següents

wal_level = replica

archive_mode = on

archive_command = 'cp %p /var/postgres/archive/%f'

wal_level = replica # minimal, replica, or logical

archive_mode = on # enables archiving; off, on, or always
(change requires restart)

archive_command = 'cp %p /var/postgres/archive/%f' # command to use to archive a WAL file
should always %p = path of file to archive

archive_command = 'cp %p /share/archive/%f & chmod 700 /share/archive/%f' # command to use to archive a WAL file

Després d'això reiniciem el servei

```
eric@serverhosp:~$ systemctl restart postgresql
==== AUTHENTICATING FOR org.freedesktop.systemd1.manage-units ====
Authentication is required to restart 'postgresql.service'.
Authenticating as: serverhosp
Password:
==== AUTHENTICATION COMPLETE ====
eric@serverhosp:~$ sudo systemctl status postgresql
● postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: enabled)
   Active: active (exited) since Wed 2026-04-29 16:13:06 UTC; 21s ago
     Process: 10195 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
    Main PID: 10195 (code=exited, status=0/SUCCESS)
       CPU: 3ms

Apr 29 16:13:06 serverhosp systemd[1]: Starting postgresql.service - PostgreSQL RDBMS...
Apr 29 16:13:06 serverhosp systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
eric@serverhosp:~$ |
```

Un cop el servei esta corrent correctament configurarem el share NFS per al node slave

Configuració share NFS

Per assegurar de millor manera els arxius crearé un usuari per que utilitzi el servidor cloud.

Aquest usuari no tindrà permís de login interactiu i només tindrà accés de lectura i execució als arxius dins del share.

```
eric@serverhosp:~$ sudo useradd pgbackup
eric@serverhosp:~$ sudo passwd pgbackup
New password:
Retype new password:
passwd: password updated successfully
eric@serverhosp:~$ |
```

Creem usuari i posem password

```
eric@serverhosp:~$ sudo usermod -s /usr/sbin/nologin pgbackup
```

li retirem el permís de login

I crearem un grup que doni permís a root i postgres per poder escriure i llegir a aquesta carpeta.

```
root@serverhosp:/home/eric# groupadd pgaccess
root@serverhosp:/home/eric# usermod -aG pgaccess postgres
root@serverhosp:/home/eric# usermod -aG pgaccess eric
root@serverhosp:/home/eric# usermod -aG pgaccess root
```

Ara crearem la carpeta que compartirem amb NFS i li assignarem permisos mínims per enfortir la seguretat.

```
root@serverhosp:/home/eric# mkdir -p /share/archive
root@serverhosp:/home/eric# chown pgbackup:pgaccess /share/archive
root@serverhosp:/home/eric# chmod 570 /share/archive
root@serverhosp:/home/eric# |
```

Després de configurar tot el sistema de carpetes passarem a muntar la carpeta compartida.

Installem el paquet que ens permet compartir carpetes.

```
eric@serverhosp:~$ sudo apt install nfs-kernel-server
```

Ara configurarem la carpeta compartida, per això necessitarem saber el id del usuari pgbakup

```
eric@serverhosp:~$ id pgbakup
uid=1002(pgbakup) gid=1002(pgbakup) groups=1002(pgbakup)
eric@serverhosp:~$ |
```

I la ip del servidor remot, com que es una VPN ho podem veure amb la comanda de tailscale

tailscale status

```
eric@serverhosp:~$ tailscale status
100.105.147.27 serverhosp lleerrriiiccc@ linux -
100.101.108.31 aiproject-vps lleerrriiiccc@ linux idle, tx 956 rx 948
eric@serverhosp:~$ |
```

o ho podem mirar al panell d'administrador de tailscale

MACHINE	ADDRESSES	VERSION	LAST SEEN	
aiproject-vps lleerrriiiccc@github	100.101.108.31	1.96.4 Linux 6.8.0-100-generic	Connected	...
serverhosp lleerrriiiccc@github	100.105.147.27	1.96.4 Linux 6.8.0-110-generic	Connected	...

I obrirem el fitxer /etc/exports

Per afegir aquesta línia

```
/share/archive 100.101.108.31(rw,sync,no_subtree_check,root_squash,all_squash,anonuid=1002,anongid=1002)
```

```
GNU nano 7.2 /etc/exports *
# /etc/exports: the access control list for filesystems which may be exported
# to NFS clients. See exports(5).
#
# Example for NFSv2 and NFSv3:
# /srv/homes hostname1(rw,sync,no_subtree_check) hostname2(ro,sync,no_subtree_check)
#
# Example for NFSv4:
# /srv/nfs4 gss/krb5i(rw,sync,fsid=0,crossmnt,no_subtree_check)
# /srv/nfs4/homes gss/krb5i(rw,sync,no_subtree_check)
#
/share/archive 100.101.108.31(rw,sync,no_subtree_check,root_squash,all_squash,anonuid=1002,anongid=1002)|
```

i aplicarem els canvis

```
eric@serverhosp:~$ sudo exportfs -v
/share/archive 100.101.108.31(sync,wdelay,hide,no_subtree_check,anonuid=1002,anongid=1002,sec=sys,rw,secure,root_squash,all_squash)
eric@serverhosp:~$ |
```

Servidor cloud

Configuració share NFS

Primer configurarem el share NFS per facilitar la feina després del traspàs de la còpia inicial.

El primer pas és crear el directori on es muntarà la carpeta al servidor remot.

```
root@aiproject-vps:~# mkdir -p /mnt/remote-archive
root@aiproject-vps:~# |
```

Ara instal·larem el paquet que ens permet muntar els shares NFS i configurarem la connexió

```
root@aiproject-vps:~# sudo apt install nfs-common
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
|
```

Necessitarem saber la IP de la màquina local, es pot veure al panell admin de tailscale o amb la comanda

```
tailscale status
```

Ara executem la comanda mount

```
root@aiproject-vps:~# sudo mount 100.105.147.27:/share/archive /mnt/remote-archive
root@aiproject-vps:~# |
```

Anem a provar els permisos, intentaré crear un arxiu, no m'ho hauria de permetre

```
root@aiproject-vps:~# touch /mnt/remote-archive/test.txt
touch: cannot touch '/mnt/remote-archive/test.txt': Permission denied
root@aiproject-vps:~# sudo touch /mnt/remote-archive/test.txt
touch: cannot touch '/mnt/remote-archive/test.txt': Permission denied
root@aiproject-vps:~# |
```


Configuració postgres

Primer agafarem una còpia sencera de la base de dades original.

```
eric@serverhosp:~$ pg_basebackup -h localhost -D /tmp/basebackup -U postgres -P -R
31755/31755 kB (100%), 1/1 tablespace
eric@serverhosp:~$ ls -l /tmp/basebackup/
total 268
-rw----- 1 eric eric    225 May  4 14:36 backup_label
-rw----- 1 eric eric 189024 May  4 14:36 backup_manifest
drwx----- 6 eric eric   4096 May  4 14:36 base
drwx----- 2 eric eric   4096 May  4 14:36 global
drwx----- 2 eric eric   4096 May  4 14:36 pg_commit_ts
drwx----- 2 eric eric   4096 May  4 14:36 pg_dynshmem
drwx----- 4 eric eric   4096 May  4 14:36 pg_logical
drwx----- 4 eric eric   4096 May  4 14:36 pg_multixact
drwx----- 2 eric eric   4096 May  4 14:36 pg_notify
drwx----- 2 eric eric   4096 May  4 14:36 pg_replslot
drwx----- 2 eric eric   4096 May  4 14:36 pg_serial
drwx----- 2 eric eric   4096 May  4 14:36 pg_snapshots
drwx----- 2 eric eric   4096 May  4 14:36 pg_stat
drwx----- 2 eric eric   4096 May  4 14:36 pg_stat_tmp
drwx----- 2 eric eric   4096 May  4 14:36 pg_subtrans
drwx----- 2 eric eric   4096 May  4 14:36 pg_tblspc
drwx----- 2 eric eric   4096 May  4 14:36 pg_twophase
-rw----- 1 eric eric     3 May  4 14:36 PG_VERSION
drwx----- 3 eric eric   4096 May  4 14:36 pg_wal
drwx----- 2 eric eric   4096 May  4 14:36 pg_xact
-rw----- 1 eric eric    401 May  4 14:36 postgresql.auto.conf
-rw----- 1 eric eric     0 May  4 14:36 standby.signal
eric@serverhosp:~$ |
```

I copiarem els continguts al servidor slave

```
eric@serverhosp:~$ scp -r /tmp/basebackup root@100.101.108.31:/var/lib/postgresql/
root@100.101.108.31's password:
Permission denied, please try again.
root@100.101.108.31's password:
backup_manifest                                100% 185KB 656.0KB/s 00:00
standby.signal                                100% 0      0.0KB/s 00:00
1247_vm                                        100% 8192   138.0KB/s 00:00
3603_fsm                                       100% 24KB 401.7KB/s 00:00
4146                                           100% 8192   143.2KB/s 00:00
2606_vm                                        100% 8192   26.5KB/s 00:00
2605_fsm                                       100% 24KB 431.3KB/s 00:00
2609                                           100% 360KB 949.2KB/s 00:00
1249_vm                                        100% 8192   136.3KB/s 00:00
2612                                           100% 8192   156.7KB/s 00:00
|
```

Ens assegurem que el servei està parat

```
root@aiproject-vps:/mnt/remote-archive# systemctl status postgresql
o postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: enabled)
   Active: inactive (dead) since Wed 2026-04-29 16:42:49 UTC; 4 days ago
     Duration: 1month 2w 5d 15h 26min 49.364s
    Main PID: 922470 (code=exited, status=0/SUCCESS)
      CPU: 3ms

Mar 10 14:45:59 aiproject-vps systemd[1]: Starting postgresql.service - PostgreSQL RDBMS...
Mar 10 14:45:59 aiproject-vps systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
Apr 29 16:42:49 aiproject-vps systemd[1]: postgresql.service: Deactivated successfully.
Apr 29 16:42:49 aiproject-vps systemd[1]: Stopped postgresql.service - PostgreSQL RDBMS.
root@aiproject-vps:/mnt/remote-archive# |
```

Ara configurem per arrancar en mode recovery

```
root@aiproject-vps:/var/lib/postgresql/16/main# ls -l
total 288
-rwx----- 1 postgres postgres    3 May  5 14:33 PG_VERSION
-rwx----- 1 postgres postgres  227 May  5 14:33 backup_label.old
-rwx----- 1 postgres postgres 189026 May  5 14:28 backup_manifest
drwx----- 6 postgres postgres  4096 May  5 14:31 base
drwx----- 2 postgres postgres  4096 May  5 14:33 basebackup
-rw----- 1 postgres postgres    44 May  5 14:36 current_logfiles
drwx----- 2 postgres postgres  4096 May  5 14:36 global
drwx----- 2 postgres postgres  4096 May  5 14:36 log
drwx----- 2 postgres postgres  4096 May  5 14:33 pg_commit_ts
drwx----- 2 postgres postgres  4096 May  5 14:33 pg_dynshmem
drwx----- 4 postgres postgres  4096 May  5 14:33 pg_logical
drwx----- 4 postgres postgres  4096 May  5 14:33 pg_multixact
drwx----- 2 postgres postgres  4096 May  5 14:33 pg_notify
drwx----- 2 postgres postgres  4096 May  5 14:33 pg_replslot
drwx----- 2 postgres postgres  4096 May  5 14:33 pg_serial
drwx----- 2 postgres postgres  4096 May  5 14:28 pg_snapshots
drwx----- 2 postgres postgres  4096 May  5 14:33 pg_stat
drwx----- 2 postgres postgres  4096 May  5 14:33 pg_stat_tmp
drwx----- 2 postgres postgres  4096 May  5 14:28 pg_subtrans
drwx----- 2 postgres postgres  4096 May  5 14:33 pg_tblspc
drwx----- 2 postgres postgres  4096 May  5 14:33 pg_twophase
drwx----- 3 postgres postgres  4096 May  5 14:33 pg_wal
drwx----- 2 postgres postgres  4096 May  5 14:33 pg_xact
-rwx----- 1 postgres postgres    88 May  5 14:36 postgresql.auto.conf
-rw----- 1 postgres postgres   130 May  5 14:36 postmaster.opts
-rw----- 1 postgres postgres   102 May  5 14:36 postmaster.pid
-rwx----- 1 postgres postgres    0 May  5 14:28 standby.signal
root@aiproject-vps:/var/lib/postgresql/16/main# |
```

Haurem de crear aquest arxiu per que arranqui en mode recovery i buidar el següent perquè no es connecti directament al servidor primari i faig servir el mètode d'arxius

```

root@aiproject-vps:/var/lib/postgresql/16/main# ls -l
total 288
-rwx----- 1 postgres postgres 3 May 5 14:33 PG_VERSION
-rwx----- 1 postgres postgres 227 May 5 14:33 backup_label.old
-rwx----- 1 postgres postgres 189026 May 5 14:28 backup_manifest
drwx----- 6 postgres postgres 4096 May 5 14:31 base
drwx----- 2 postgres postgres 4096 May 5 14:33 basebackup
-rw----- 1 postgres postgres 44 May 5 14:36 current_logfiles
drwx----- 2 postgres postgres 4096 May 5 14:36 global
drwx----- 2 postgres postgres 4096 May 5 14:36 log
drwx----- 2 postgres postgres 4096 May 5 14:33 pg_commit_ts
drwx----- 2 postgres postgres 4096 May 5 14:33 pg_dynshmem
drwx----- 4 postgres postgres 4096 May 5 14:33 pg_logical
drwx----- 4 postgres postgres 4096 May 5 14:33 pg_multixact
drwx----- 2 postgres postgres 4096 May 5 14:33 pg_notify
drwx----- 2 postgres postgres 4096 May 5 14:33 pg_replslot
drwx----- 2 postgres postgres 4096 May 5 14:33 pg_serial
drwx----- 2 postgres postgres 4096 May 5 14:28 pg_snapshots
drwx----- 2 postgres postgres 4096 May 5 14:33 pg_stat
drwx----- 2 postgres postgres 4096 May 5 14:33 pg_stat_tmp
drwx----- 2 postgres postgres 4096 May 5 14:28 pg_subtrans
drwx----- 2 postgres postgres 4096 May 5 14:33 pg_tblspc
drwx----- 2 postgres postgres 4096 May 5 14:33 pg_twophase
drwx----- 3 postgres postgres 4096 May 5 14:33 pg_wal
drwx----- 2 postgres postgres 4096 May 5 14:33 pg_xact
-rwx----- 1 postgres postgres 88 May 5 14:36 postgresql.auto.conf
-rw----- 1 postgres postgres 130 May 5 14:36 postmaster.opts
-rw----- 1 postgres postgres 102 May 5 14:36 postmaster.pid
-rwx----- 1 postgres postgres 0 May 5 14:28 standby.signal
root@aiproject-vps:/var/lib/postgresql/16/main# |

```

Ara configurarem postgres per fer la replicació

```

# Primary node name
hot_standby = on
# "off" disallows queries during recovery

```

Amb aquesta línia configurem des de on agafa els arxius de WAL

```

# These are only used in recovery mode.
restore_command = 'cp /mnt/remote-archive/%f %p'
# command to use to restore an archived WAL file

```

Ara arranquem el servei

```

root@aiproject-vps:/var/lib/postgresql/16/main# systemctl start postgresql
root@aiproject-vps:/var/lib/postgresql/16/main# systemctl status postgresql
● postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: enabled)
   Active: active (exited) since Tue 2026-05-05 14:36:26 UTC; 5min ago
     Process: 658394 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
    Main PID: 658394 (code=exited, status=0/SUCCESS)
      CPU: 3ms

May 05 14:36:26 aiproject-vps systemd[1]: Starting postgresql.service - PostgreSQL RDBMS...
May 05 14:36:26 aiproject-vps systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
root@aiproject-vps:/var/lib/postgresql/16/main# |

```


I mirem els logs per corroborar que ha fet la replicació

```
2026-05-05 14:36:24.055 UTC [658319] DEBUG: end of backup record reached
2026-05-05 14:36:24.055 UTC [658319] CONTEXT: WAL redo at 0/170000D8 for XLOG/BACKUP_END: 0/17000028
2026-05-05 14:36:24.055 UTC [658319] DEBUG: end of backup reached
2026-05-05 14:36:24.055 UTC [658319] LOG: completed backup recovery with redo LSN 0/17000028 and end LSN 0/17000100
2026-05-05 14:36:24.056 UTC [658319] LOG: consistent recovery state reached at 0/17000100
2026-05-05 14:36:24.056 UTC [658315] LOG: database system is ready to accept read-only connections
```

Ara ja tenim la redundància activa.

Comprovació

Ara forçaré un arxiu WAL per comprovar que es completar correctament la operació.

```
postgres=# SELECT pg_switch_wal();
pg_switch_wal
-----
0/3C000160
(1 row)

postgres=# |
```

Carpeta servidor local

```
postgres=# \q
root@serverhosp:/tmp/basebackup# ls -l /share/archive/
total 344084
-rwx-----+ 1 postgres postgres 16777216 May  6 17:15 00000001000000000000000028
-rwx-----+ 1 postgres postgres 16777216 May  6 17:15 00000001000000000000000029
-rwx-----+ 1 postgres postgres      341 May  6 17:15 00000001000000000000000029.00000028.backup
-rwx-----+ 1 postgres postgres 16777216 May  6 17:22 000000010000000000000000002A
-rwx-----+ 1 postgres postgres 16777216 May  7 13:49 000000010000000000000000002B
-rwx-----+ 1 postgres postgres 16777216 May  7 13:49 000000010000000000000000002C
-rwx-----+ 1 postgres postgres      341 May  7 13:49 000000010000000000000000002C.00000028.backup
-rwx-----+ 1 postgres postgres 16777216 May  7 14:39 000000010000000000000000002D
-rwx-----+ 1 postgres postgres 16777216 May  7 14:41 000000010000000000000000002E
-rwx-----+ 1 postgres postgres 16777216 May  7 14:42 000000010000000000000000002F
-rwx-----+ 1 postgres postgres 16777216 May  7 14:42 0000000100000000000000000030
-rwx-----+ 1 postgres postgres      341 May  7 14:42 0000000100000000000000000030.00000028.backup
-rwx-----+ 1 postgres postgres 16777216 May  7 14:43 0000000100000000000000000031
-rwx-----+ 1 postgres postgres 16777216 May  7 14:43 0000000100000000000000000032.00000028.backup
-rwx-----+ 1 postgres postgres      341 May  7 14:43 0000000100000000000000000032.00000028.backup
-rwx-----+ 1 postgres postgres 16777216 May  7 14:57 0000000100000000000000000033
-rwx-----+ 1 postgres postgres 16777216 May  7 14:57 0000000100000000000000000034
-rwx-----+ 1 postgres postgres 16777216 May  7 15:46 0000000100000000000000000035
-rwx-----+ 1 postgres postgres 16777216 May  7 15:47 0000000100000000000000000036
-rwx-----+ 1 postgres postgres 16777216 May  7 15:48 0000000100000000000000000037
-rwx-----+ 1 postgres postgres 16777216 May  7 15:49 0000000100000000000000000038
-rwx-----+ 1 postgres postgres      341 May  7 15:49 0000000100000000000000000038.00000028.backup
-rwx-----+ 1 postgres postgres 16777216 May  7 15:55 0000000100000000000000000039
-rwx-----+ 1 postgres postgres 16777216 May  7 16:02 000000010000000000000000003A
-rwx-----+ 1 postgres postgres 16777216 May  7 16:02 000000010000000000000000003B
-rwx-----+ 1 postgres postgres 16777216 May  7 16:13 000000010000000000000000003C
root@serverhosp:/tmp/basebackup# |
```

Carpeta servidor cloud

```
root@aiproject-vps:/var/lib/postgresql/16/main# ls -l /mnt/remote-archive/
total 344084
-rwx-----+ 1 postgres netdev 16777216 May 6 17:15 000000010000000000000028
-rwx-----+ 1 postgres netdev 16777216 May 6 17:15 000000010000000000000029
-rwx-----+ 1 postgres netdev 341 May 6 17:15 000000010000000000000029.00000028.backup
-rwx-----+ 1 postgres netdev 16777216 May 6 17:22 00000001000000000000002A
-rwx-----+ 1 postgres netdev 16777216 May 7 13:49 00000001000000000000002B
-rwx-----+ 1 postgres netdev 16777216 May 7 13:49 00000001000000000000002C
-rwx-----+ 1 postgres netdev 341 May 7 13:49 00000001000000000000002C.00000028.backup
-rwx-----+ 1 postgres netdev 16777216 May 7 14:39 00000001000000000000002D
-rwx-----+ 1 postgres netdev 16777216 May 7 14:41 00000001000000000000002E
-rwx-----+ 1 postgres netdev 16777216 May 7 14:42 00000001000000000000002F
-rwx-----+ 1 postgres netdev 16777216 May 7 14:42 000000010000000000000030
-rwx-----+ 1 postgres netdev 341 May 7 14:42 000000010000000000000030.00000028.backup
-rwx-----+ 1 postgres netdev 16777216 May 7 14:43 000000010000000000000031
-rwx-----+ 1 postgres netdev 16777216 May 7 14:43 000000010000000000000032
-rwx-----+ 1 postgres netdev 341 May 7 14:43 000000010000000000000032.00000028.backup
-rwx-----+ 1 postgres netdev 16777216 May 7 14:57 000000010000000000000033
-rwx-----+ 1 postgres netdev 16777216 May 7 14:57 000000010000000000000034
-rwx-----+ 1 postgres netdev 16777216 May 7 15:46 000000010000000000000035
-rwx-----+ 1 postgres netdev 16777216 May 7 15:47 000000010000000000000036
-rwx-----+ 1 postgres netdev 16777216 May 7 15:48 000000010000000000000037
-rwx-----+ 1 postgres netdev 16777216 May 7 15:49 000000010000000000000038
-rwx-----+ 1 postgres netdev 341 May 7 15:49 000000010000000000000038.00000028.backup
-rwx-----+ 1 postgres netdev 16777216 May 7 15:55 000000010000000000000039
-rwx-----+ 1 postgres netdev 16777216 May 7 16:02 00000001000000000000003A
-rwx-----+ 1 postgres netdev 16777216 May 7 16:02 00000001000000000000003B
-rwx-----+ 1 postgres netdev 16777216 May 7 16:13 00000001000000000000003C
```

Logs del servidor postgres cloud

```
2026-05-07 16:13:12.295 UTC [1402680] LOG: waiting for WAL to become available at 0/3C000018
2026-05-07 16:13:25.625 UTC [1402680] LOG: restored log file "00000001000000000000003C" from archive
2026-05-07 16:13:25.637 UTC [1402680] DEBUG: got WAL segment from archive
cp: cannot stat '/mnt/remote-archive/00000001000000000000003D': No such file or directory
```

Automatització davant caiguda

Preparació

Al equip remot crearem un usuari amb contrasenya perquè es connecti per ssh i aixequi el servidor en cas de fallada del principal.

```
root@aiproject-vps:/var/lib/postgresql/16/main# sudo useradd crash_user
root@aiproject-vps:/var/lib/postgresql/16/main# passwd crash_user
New password:
Retype new password:
passwd: password updated successfully
```

Després configurarem ssh per acceptar connexions amb certificat i automatitzar el procés de connexió.

```
eric@serverhosp:~/dev$ ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/eric/.ssh/id_ed25519):
Created directory '/home/eric/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/eric/.ssh/id_ed25519
Your public key has been saved in /home/eric/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:RgP9qPrUwhCKN3+wfYsd7Lw3yaWizmXkP0HTGJA2E eric@serverhosp
The key's randomart image is:
+--[ED25519 256]--+
| .. E. |
| ..... |
| . oo . . |
| . . . . . o + |
| . + o .S . o |
| . o *. * . o |
| . o * O. B . . |
| . + X.+B + . |
| . o *. *+. + |
+-----[SHA256]-----+
```

```

eric@serverhosp:~/dev$ ssh-copy-id crash_user@100.101.108.31
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/eric/.ssh/id_ed25519.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
crash_user@100.101.108.31's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'crash_user@100.101.108.31'"
and check to make sure that only the key(s) you wanted were added.

eric@serverhosp:~/dev$ |

```

Ara si executem la conexio no ens hauria de demanar contrasenya

```

eric@serverhosp:~/dev$ ssh crash_user@100.101.108.31
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-100-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu May  7 16:34:18 UTC 2026

System load:  0.14               Processes:            159
Usage of /:   50.1% of 47.39GB   Users logged in:     1
Memory usage: 80%               IPv4 address for eth0: 159.65.197.160
Swap usage:   0%                IPv4 address for eth0: 10.18.0.5

Expanded Security Maintenance for Applications is not enabled.

47 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

5 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

*** System restart required ***
$ echo $SHELL
/bin/sh
$ |

```

Script

La estructura de scripts es la següent, un cron que executa cada dos minuts si postgres principal està viu, si no ho està executa un script del servidor secundari que el posa en marxa i el treu del mode recuperació.

Script node principal

```
GNU nano 7.2
#!/bin/bash

SERVER="100.101.108.31"
USER="crash_user"

state=$(systemctl is-active postgresql)

if [ "$state" = "inactive" ]; then
    ssh $USER@$SERVER "sudo failover"
```

Script node secundari

```
GNU nano 7.2 failover.sh *
#!/bin/bash
sudo systemctl stop postgresql
sudo mv /var/lib/postgresql/16/main/standby.signal /var/lib/postgresql/16/main/standby.signal.bck
sudo systemctl start postgresql
```

Un cop creat l'script el que fem es convertir-lo en un executable del sistema

```
root@aiproject-vps:/home/dev# ls -l
total 4
-rwxr-xr-x 1 root root 173 May  7 16:46 failover.sh
root@aiproject-vps:/home/dev# cp failover.sh /bin/failover
root@aiproject-vps:/home/dev# failover
root@aiproject-vps:/home/dev# |
```

Tasca de cron

```
# m h dom mon dow  command
*/2 * * * * /home/eric/dev/crash_auto.sh
```


Configurem la tasca cada dos minuts al node principal per executar l'script de comprovació.

Prova de concepte

Ara programar una tasca amb at per apagar el servei de postgres al node principal i així veure si funciona correctament el sistema autònom de failover.

```
May 07 17:22:01 serverhosp CRON[3609]: pam_unix(cron:session): session opened for user eric(uid=1001) by eric(uid=0)
May 07 17:22:01 serverhosp CRON[3610]: (eric) CMD (/home/eric/dev/crash_auto.sh)
May 07 17:22:04 serverhosp CRON[3609]: (CRON) info (No MTA installed, discarding output)
May 07 17:22:04 serverhosp CRON[3609]: pam_unix(cron:session): session closed for user eric
```

Execució de cron

```
May 07 17:22:03 aiproject-vps sudo[1427050]: crash_user : PWD=/home/crash_user ; USER=root ; COMMAND=/bin/failover
```

Comanda executada al servidor remot

```
-rwx----- 1 postgres postgres 0 May 7 15:50 standby.signal.bck
root@aiproject-vps:/home/dev# |
```

```
root@aiproject-vps:~# journalctl -f | grep failover
May 11 13:20:03 aiproject-vps sudo[2806029]: crash_user : PWD=/home/crash_user ; USER=root ; COMMAND=/bin/failover
```

```
root@aiproject-vps:~# systemctl status postgresql
○ postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: enabled)
   Active: inactive (dead) since Mon 2026-05-11 13:20:08 UTC; 59s ago
   Duration: 3.560s
   Process: 2806049 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
   Main PID: 2806049 (code=exited, status=0/SUCCESS)
   CPU: 9ms

May 11 13:20:04 aiproject-vps systemd[1]: Starting postgresql.service - PostgreSQL RDBMS...
May 11 13:20:04 aiproject-vps systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
May 11 13:20:08 aiproject-vps systemd[1]: postgresql.service: Deactivated successfully.
May 11 13:20:08 aiproject-vps systemd[1]: Stopped postgresql.service - PostgreSQL RDBMS.
root@aiproject-vps:~# |
```

Proposta model de backups

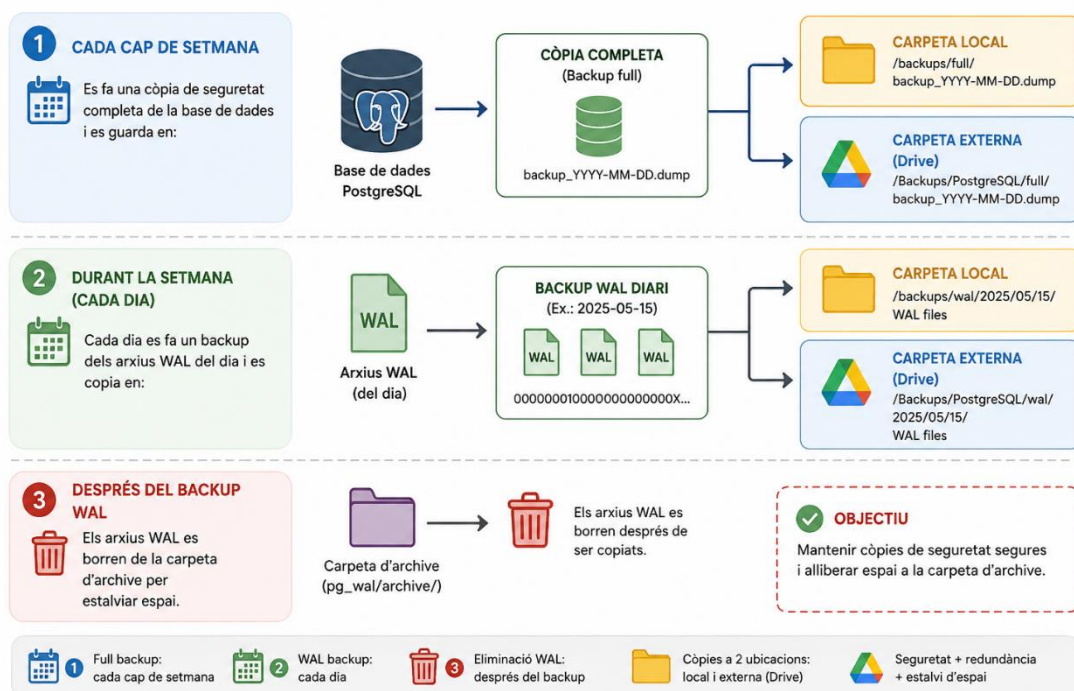
Pla de còpies de seguretat

Davant la possibilitat d'un incident que requereixi la recuperació de dades hem dissenyat aquest pla de recuperació.

Tenim en compte que ja esta implementat el funcionament amb arxius WAL aprofitarem aquesta eina per els nostres backups.

Aquí tenim un esquema visual del procediment de backups.

ESQUEMA DEL PLA DE CÒPIES DE SEGURETAT – POSTGRESQL



Es farà un backup de la base de dades sencera setmanalment durant el cap de setmana durant la nit a les 02:00 tenint en compte que es la hora amb menys activitat, d'igual manera aquesta hora es pot adaptar segons les característiques del hospital.

Aquesta copia es guardarà de manera local dins del mateix servidor i s'externalitzara en una carpeta de drive. Aquesta tasca s'executarà de manera automàtica amb una tasca programada de cron i un script de Python. D'aquesta manera evitem fer còpies completes a diari mantenint una consistència de còpies relativament curta.

Durant la setmana a diari durant la matinada es farà un backup dels arxius WAL mantenint la capacitat de recuperar dades diària. De la mateixa manera es guarda una copia al mateix servidor i una a la carpeta externa de drive.

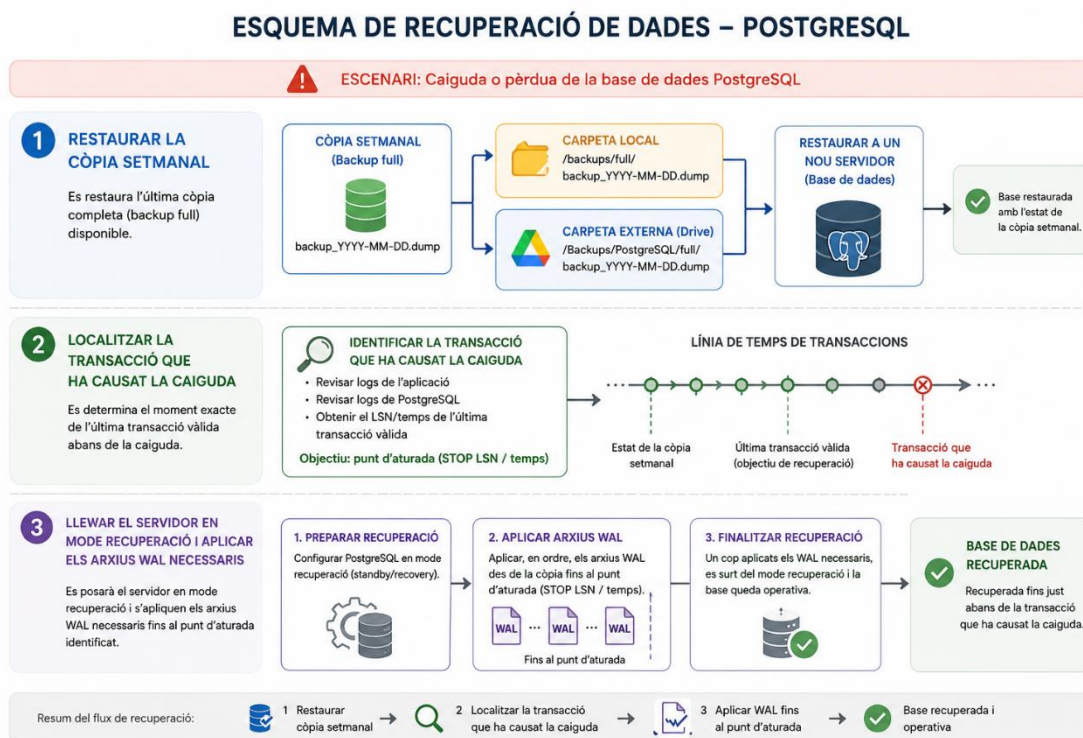
Per estalviar espai de disc els arxius WAL ja raspallats es borran de la carpeta compartida on estan per sincronitzar amb el servidor remot.

Pla d'actuació

Davant la caiguda de la base de dades el pla de restauració es el següent.

- 1 - Restaurar la copia setmanal corresponent.
- 2 - Analitzar logs i localitzar la transacció problemàtica.
- 3 - Aplicar arxius WAL fins a la transacció problemàtica.

Aquí un diagrama visual.



Bloc de consultes

Explicació general

En aquest bloc del projecte se'ns encarrega la tasca de generar diferents informes sobre la situació del hospital com per exemple: malalties mes comunes, habitacions per planta...

Aquest bloc es troba el codi dins el [GITHUB](#).

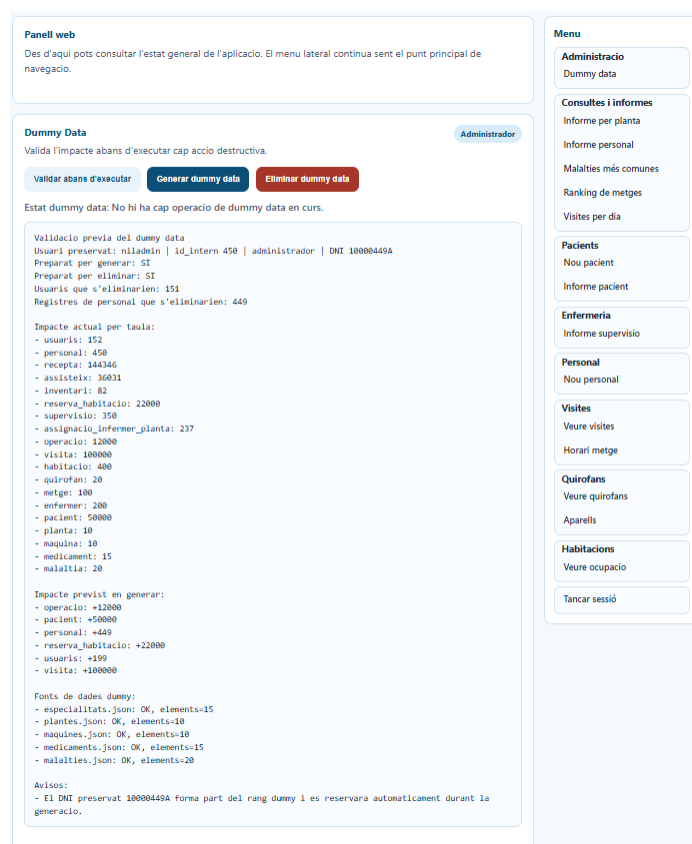
Dummy Data

Disseny general de la solució

La generació del Dummy data s'ha centralitzat en un servei específic del backend, de manera que tota la lògica de creació, validació i eliminació queda encapsulada en un únic punt. Aquesta decisió facilita el manteniment del codi, evita duplicacions i permet reutilitzar la mateixa lògica tant des del client d'escriptori com des de la interfície web.

A nivell funcional, el flux de treball és el següent. Un usuari administrador accedeix al menú de Dummy Data. Abans d'executar res, pot consultar una validació prèvia que comprova l'estat de la base de dades, la disponibilitat dels fitxers font necessaris i l'impacte aproximat de l'operació. Si tot és correcte, l'usuari pot llançar la generació o l'eliminació. L'execució es fa en segon pla per no bloquejar la interfície i el sistema va mostrant l'estat de l'operació fins que finalitza.

Aquesta mateixa operativa s'ha integrat tant a l'aplicació d'escriptori com a la versió web. D'aquesta manera, la funcionalitat queda accessible des dels dos canals d'ús del projecte i el comportament és coherent en ambdós entorns.



Generació de les dades

La generació de dades es fa per fases, respectant l'ordre lògic de les dependències entre taules. Primer es carreguen els catàlegs base, com ara plantes, malalties, medicaments, màquines i especialitats. Aquestes dades provenen de fitxers estructurats i serveixen de base per construir després la resta d'informació relacional.

A continuació es genera el personal del centre. En aquesta fase es creen 100 metges, 200 infermers, 100 perfils de suport i neteja i 50 perfils d'administració. En la implementació actual, les persones de neteja es modelen dins el tipus de personal tècnic. Quan la generació s'executa des de l'aplicació, es conserva l'administrador autenticat, de manera

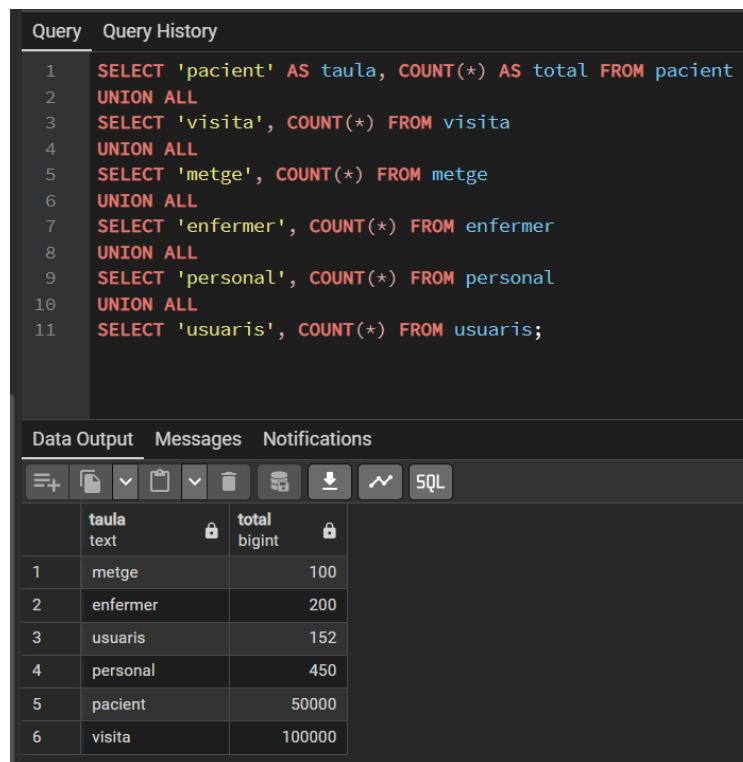
que el sistema crea 49 administradors nous i manté el preservat, assolint igualment el total final de 50 administradors requerits. Això evita duplicar identitats i impedeix perdre l'accés administratiu després de la regeneració.

Després es creen els pacients, amb un total de 50.000 registres. Cada pacient té un identificador únic, un nom, cognoms i una data de naixement coherent. Tot seguit es generen les àrees físiques del sistema, com les habitacions, els quiròfans, l'inventari de màquines i les assignacions d'infermers a plantes.

Un cop existeixen les entitats bàsiques, es creen 100.000 visites. Cada visita vincula un pacient, un metge i una malaltia, i incorpora una data, una hora i un diagnòstic relacionat amb la patologia assignada. També es generen receptes associades a una part significativa de les visites per fer la càrrega de dades més realista. Addicionalment, s'han inclòs operacions quirúrgiques, equips d'assistència i reserves d'habitacions per enriquir el conjunt de dades i poder exercitar millor els informes i consultes del sistema.

Per optimitzar el rendiment de la càrrega, les insercions massives es realitzen per blocs, evitant fer una inserció individual per cada registre. Aquesta estratègia redueix el nombre de viatges entre aplicació i base de dades i millora clarament el temps total de generació.

Després de la generació de totes les dades la base de dades quedaria així:



The screenshot shows a database query tool interface. The top section, titled 'Query', contains a SQL query that counts the number of records in several tables. The query is as follows:

```
1 SELECT 'pacient' AS taula, COUNT(*) AS total FROM pacient
2 UNION ALL
3 SELECT 'visita', COUNT(*) FROM visita
4 UNION ALL
5 SELECT 'metge', COUNT(*) FROM metge
6 UNION ALL
7 SELECT 'enfermer', COUNT(*) FROM enfermer
8 UNION ALL
9 SELECT 'personal', COUNT(*) FROM personal
10 UNION ALL
11 SELECT 'usuaris', COUNT(*) FROM usuaris;
```

The bottom section, titled 'Data Output', shows the results of the query in a table format. The table has two columns: 'taula' (table name) and 'total' (count). The results are as follows:

	taula	total
1	metge	100
2	enfermer	200
3	usuaris	152
4	personal	450
5	pacient	50000
6	visita	100000

Consistència i qualitat del Dummy data

Un dels punts clau de la implementació és que les dades no només siguin moltes, sinó també coherents. Per aconseguir-ho, la generació respecta totes les relacions entre taules i crea els registres en un ordre compatible amb les claus foranes. Per exemple, les visites només es creen quan ja existeixen pacients, metges, malalties i medicaments; les operacions només es generen quan ja existeixen quiròfans, pacients i metges; i les assignacions d'infermers només es creen quan ja hi ha plantes i infermers disponibles.

També s'ha tingut en compte la coherència del format dels camps. Els DNIs del personal es generen amb un patró vàlid i únic, els identificadors dels pacients segueixen un format homogeni, els telèfons i correus electrònics es creen amb una estructura plausible, i les dates de naixement i d'alta es mouen dins de marges realistes. Això permet provar l'aplicació amb dades que tenen una aparença propera a la realitat i que encaixen amb les validacions del model.

Pel que fa a la internacionalització de les dades, s'ha reservat una petita proporció de noms i cognoms en alfabet ciríl·lic. Aquesta mostra és suficient per demostrar que el sistema és capaç de gestionar caràcters no llatins i per comprovar que els llistats, informes i processos de serialització tracten correctament textos amb un alfabet diferent. En la implementació s'ha fixat una proporció aproximada de l'1,5% d'aquest tipus de registres.

A més, s'ha corregit un punt crític relacionat amb la preservació de l'administrador autènticat. Inicialment podia donar-se el cas que el DNI del compte preservat coincidís amb el rang de DNIs generats automàticament. Aquesta situació s'ha resolt reservant explícitament el DNI de l'usuari preservat i exclouent-lo del conjunt de valors que es generen, de manera que la funcionalitat pot executar-se sempre sense col·lisions.

Aquí adjuntem una captura on es mostren alguns registres amb noms i cognoms en alfabet ciríl·lic:

Query

Query History

1

SELECT nom, cognom, cognom2, dni

2

FROM personal

3

WHERE nom ~ '[А-Яа-я]' OR cognom ~ '[А-Яа-я]' OR cognom2 ~ '[А-Яа-я]'

4

LIMIT 10;

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 10

	nom character varying (50)	cognom character varying (50)	cognom2 character varying (50)	dni character varying (20)
1	Андрей	Соколов	Кириллов	10000110D
2	Ксения	Воронцова	Ефремов	10000113N
3	Прасковья	Аксенова	Ефимова	10000183J
4	Регина	Зайцева	Матвеев	10000231S
5	Андрон	Лыткин	Кудряшова	10000267M
6	Аггей	Дмитриев	Абрамова	10000314Y
7	Гостомысл	Третьякова	Белова	10000348V
8	Панкрат	Анисимова	Сазонова	10000357A
9	Эраст	Артемьева	Рогова	10000406Y

Índexs i rendiment

Com que el volum de dades és elevat, la part de rendiment no depèn només de la generació massiva, sinó també de la disponibilitat d'índexs adequats per a les consultes més habituals. En aquest projecte s'han utilitzat tant índexs explícits com índexs implícits derivats de claus primàries i restriccions d'unicitat.

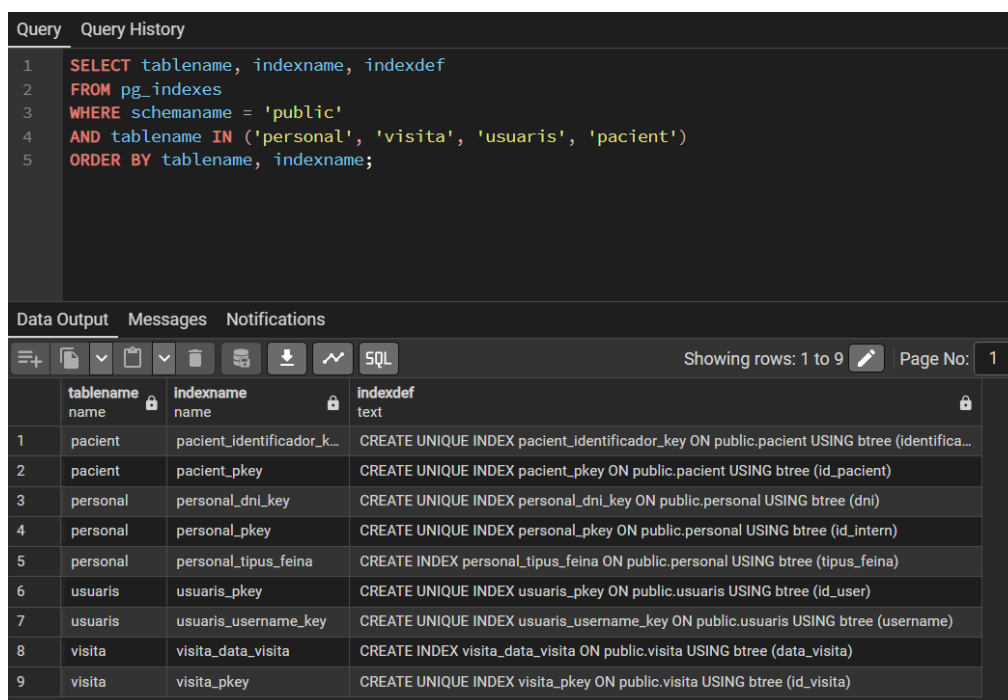
Entre els índexs explícits destaca l'índex sobre el camp tipus_feina de la taula personal. Aquest índex és útil perquè moltes operacions d'autenticació, control d'accés i informes filtren per tipus de personal, especialment quan cal distingir administradors, metges o infermers. També s'ha definit un índex sobre la data de visita, ja que els informes per rang temporal i les consultes de visites per dia són especialment rellevants en un entorn amb 100.000 registres. Finalment, s'ha creat un índex sobre el camp id_malaltia de la taula visita, que resulta especialment útil per a informes estadístics i agregacions sobre les malalties més freqüents.

A banda d'aquests, moltes taules ja disposen d'índexs implícits gràcies a les seves claus primàries i restriccions UNIQUE. És el cas del nom d'usuari a la taula d'usuaris, del DNI a la taula de personal, de l'identificador del pacient, o de les claus compostes de les taules

de relació. En aquests casos no s'han afegit índexs addicionals perquè les estructures ja existents cobreixen els accessos principals i, si se n'afegissin més sense necessitat, el cost d'inserció massiva augmentaria innecessàriament.

Per tant, la selecció final d'índexs s'ha fet equilibrant dos factors. D'una banda, la necessitat d'accelerar consultes freqüents sobre conjunts de dades grans. De l'altra, la necessitat de no penalitzar excessivament la càrrega massiva del Dummy data.

Captura on es veu tots els índex implementats dintre de la base de dades:



```
1 SELECT tablename, indexname, indexdef
2 FROM pg_indexes
3 WHERE schemaname = 'public'
4 AND tablename IN ('personal', 'visita', 'usuaris', 'pacient')
5 ORDER BY tablename, indexname;
```

	tablename name	indexname name	indexdef text
1	pacient	pacient_identificador_k...	CREATE UNIQUE INDEX pacient_identificador_key ON public.pacient USING btree (identifica...
2	pacient	pacient_pkey	CREATE UNIQUE INDEX pacient_pkey ON public.pacient USING btree (id_pacient)
3	personal	personal_dni_key	CREATE UNIQUE INDEX personal_dni_key ON public.personal USING btree (dni)
4	personal	personal_pkey	CREATE UNIQUE INDEX personal_pkey ON public.personal USING btree (id_intern)
5	personal	personal_tipus_feina	CREATE INDEX personal_tipus_feina ON public.personal USING btree (tipus_feina)
6	usuaris	usuaris_pkey	CREATE UNIQUE INDEX usuaris_pkey ON public.usuaris USING btree (id_user)
7	usuaris	usuaris_username_key	CREATE UNIQUE INDEX usuaris_username_key ON public.usuaris USING btree (username)
8	visita	visita_data_visita	CREATE INDEX visita_data_visita ON public.visita USING btree (data_visita)
9	visita	visita_pkey	CREATE UNIQUE INDEX visita_pkey ON public.visita USING btree (id_visita)

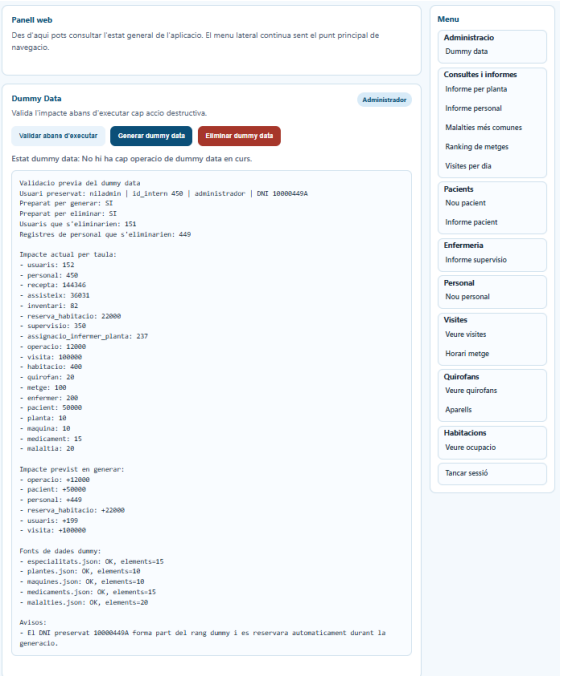
Integració dins l'aplicació

La funcionalitat s'ha integrat dins el menú principal de l'aplicació, de manera visible només per a usuaris amb perfil d'administrador. Aquesta restricció és necessària perquè tant la generació com l'eliminació del Dummy data són operacions de gran impacte i no haurien d'estar disponibles per a perfils assistencials o consultius.

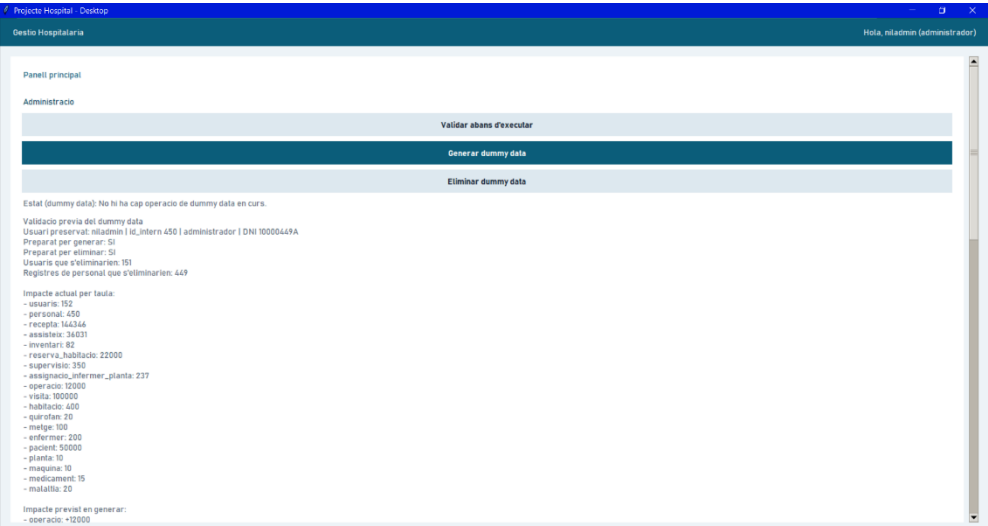
En el client d'escriptori s'ha afegit un bloc específic d'administració dins la pantalla principal. Aquest bloc inclou tres accions. La primera és una validació prèvia que informa del que passarà abans d'executar res. La segona permet iniciar la generació del Dummy data. La tercera permet eliminar-lo, conservant únicament l'usuari administrador autenticat. El panell també mostra l'estat de l'operació i s'actualitza periòdicament mentre el procés està en curs.

A la versió web s’ha replicat el mateix comportament. Dins la pàgina principal apareix un panell de Dummy Data per als administradors, i al menú lateral s’ha afegit una entrada específica d’administració. D’aquesta manera, la funcionalitat és coherent entre interfícies i no queda limitada a un únic canal d’accés.

WEB:



Tkinter:



Validació prèvia i mesures de seguretat

Com que la generació del Dummy data pot esborrar les dades existents, s'ha implementat una capa de validació prèvia per reduir riscos. Aquesta validació comprova que l'usuari autènticat continua existint i que realment és administrador, que les taules necessàries són presents, que els fitxers font de dades Dummy existeixen i que no hi ha una altra operació en curs.

A més, la validació mostra informació útil abans d'executar l'acció, com ara el nombre d'usuaris i registres de personal que s'eliminarien, el recompte actual de cada taula, l'impacte previst de la generació i els avisos o bloquejos detectats. Aquesta capa aporta traçabilitat i fa que la decisió d'executar la regeneració no sigui cega, sinó informada.

També s'ha implementat un sistema d'estat de tasca en segon pla. Això permet que el procés s'executi de manera asíncrona i que la interfície pugui mostrar missatges com Preparant operació, Generant dades o Eliminant dades, sense quedar-se bloquejada fins al final.

Captura de exemple on es pot veure el tipus d'informació que et dona la validació prèvia.

Estat (dummy data): No hi ha cap operació de dummy data en curs.

Validació prèvia del dummy data

Usuari preservat: niladmin | id_intern 450 | administrador | DNI 10000449A

Preparat per generar: SI

Preparat per eliminar: SI

Usuaris que s'eliminarien: 151

Registres de personal que s'eliminarien: 449

Eliminació del Dummy data

L'aplicació no només permet generar dades de prova, sinó també eliminar-les. Aquesta segona funcionalitat és igualment important perquè facilita tornar a un estat controlat després d'una prova de càrrega o d'una bateria de validacions. L'eliminació es realitza buidant les taules relacionades amb el conjunt Dummy, esborrant els usuaris i registres de personal no preservats i reajustant les seqüències internes de la base de dades.

De nou, es conserva l'usuari administrador que ha iniciat l'operació. Això evita deixar el sistema sense cap compte actiu amb què es pugui continuar treballant. L'usuari pot revisar l'impacte de l'eliminació abans d'executar-la i, un cop iniciada, la interfície també mostra el progrés i l'estat final.

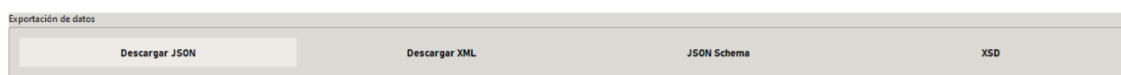
Bloc d'exportació i validació de dades

Objectiu d'aquesta funcionalitat

En aquest apartat del projecte hem implementat una funcionalitat per exportar les visites mèdiques que hi ha hagut entre dues dates. L'objectiu és que l'usuari pugui consultar un període concret i descarregar aquesta informació en un format estructurat, tant en JSON com en XML.

Aquesta funció és útil perquè permet guardar les dades fora de l'aplicació i reutilitzar-les després, per exemple per revisar-les, validar-les o compartir-les amb altres sistemes.

Aquí es pot veure el panell de exportació de dades dins de l'aplicació:

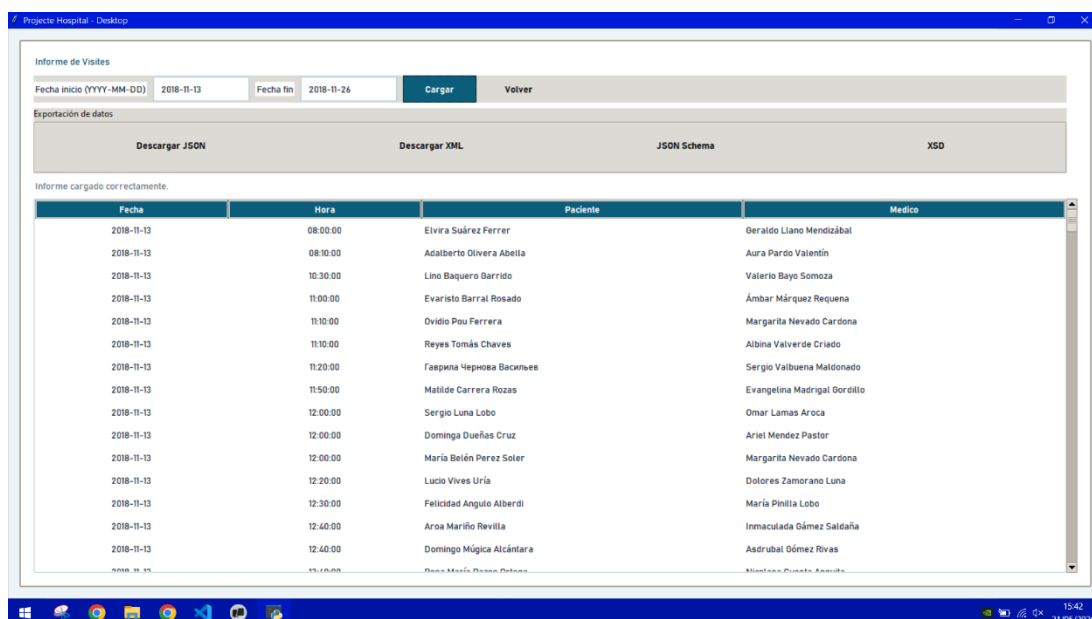


Funcionament general

Dins de l'informe de visites, l'usuari pot indicar una data d'inici i una data de fi. Un cop seleccionat aquest rang, el sistema busca totes les visites que estan compreses entre aquestes dues dates.

Després de fer la consulta, l'aplicació ofereix diferents opcions de descàrrega. L'usuari pot baixar les dades en format JSON o en format XML. A més, també pot descarregar els fitxers d'esquema corresponents per validar que l'estructura del document sigui correcta.

Aquí podem veure com posem unes dates i es carreguen les visites dins d'aquest període de temps.



Dades que conté l'exportació

Per a cada visita exportada es guarda la informació principal necessària per identificar-la i entendre-la. En concret, cada registre inclou l'identificador de la visita, el dia de la visita, el metge que ha atès el pacient i un bloc amb les dades del pacient.

En la implementació actual del projecte, dins del bloc del pacient apareixen l'identificador intern del pacient, el nom i els cognoms. D'aquesta manera, cada visita queda relacionada amb la persona atesa i amb el professional sanitari corresponent.

A més, el document exportat també incorpora el rang de dates consultat. Això fa que el fitxer sigui més clar, ja que es pot veure fàcilment de quin període s'han extret les dades.

Aquí es pot veure la estructura de cada visita i el pacient en format JSON:

```
{
  "id_visita": 68064,
  "dia": "2018-11-13",
  "metge": "Geraldo Llano Mendizábal",
  "pacient": {
    "id_pacient": 42713,
    "nom": "Elvira",
    "cognom": "Suárez",
    "cognom2": "Ferrer"
  }
},
```

Exportació en format JSON

Un dels formats disponibles és JSON. Aquest format és molt utilitzat perquè és fàcil de llegir i també és fàcil de processar des de programes i aplicacions.

Quan l'usuari descarrega les visites en JSON, el sistema genera un document estructurat i ordenat. El fitxer es guarda amb indentació utilitzant tabulacions, de manera que el resultat és més net i llegible.

Aquí es pot veure l'estructura completa del JSON:

```
1 {  
2   "data_inici": "2018-11-13",  
3   "data_fi": "2018-11-26",  
4   "visites": [  
5     {  
6       "id_visita": 68064,  
7       "dia": "2018-11-13",  
8       "metge": "Geraldo Llano Mendizábal",  
9       "pacient": {  
10        "id_pacient": 42713,  
11        "nom": "Elvira",  
12        "cognom": "Suárez",  
13        "cognom2": "Ferrer"  
14      }  
15    },  
16    {  
17      "id_visita": 33154,  
18      "dia": "2018-11-13",  
19      "metge": "Aura Pardo Valentín",  
20      "pacient": {  
21        "id_pacient": 39532,  
22        "nom": "Adalberto",  
23        "cognom": "Olivera",  
24        "cognom2": "Abella"  
25      }  
26    },  
27    {  
28      "id_visita": 3833,  
29      "dia": "2018-11-13",  
30      "metge": "Valerio Bayo Somoza",  
31      "pacient": {  
32        "id_pacient": 25278,  
33        "nom": "Lino",  
34        "cognom": "Baquero",  
35        "cognom2": "Garrido"  
36      }  
37    }  
38  ],  
39 }
```

Exportació en format XML

L'altra opció disponible és XML. En aquest cas, el sistema genera un document jeràrquic amb una etiqueta arrel i diferents etiquetes internes per representar el rang de dates i la llista de visites.

Igual que passa amb el JSON, el fitxer XML també es genera amb indentació per tabulacions. Això facilita molt la seva lectura i permet veure clarament l'estructura del document.

Aquí podem veure una captura de la estructura del arxiu que es guarda XML:

```
1 <?xml version='1.0' encoding='utf-8'?>  
2 <exportacio_visites>  
3   <rang_dates>  
4     <data_inici>2018-11-13</data_inici>  
5     <data_fi>2018-11-26</data_fi>  
6   </rang_dates>  
7   <visites>  
8     <visita>  
9       <id_visita>68064</id_visita>  
10      <dia>2018-11-13</dia>  
11      <metge>Geraldo Llano Mendizábal</metge>  
12      <pacient>  
13        <id_pacient>42713</id_pacient>  
14        <nom>Elvira</nom>  
15        <cognom>Suárez</cognom>  
16        <cognom2>Ferrer</cognom2>  
17      </pacient>  
18    </visita>  
19    <visita>  
20      <id_visita>33154</id_visita>  
21      <dia>2018-11-13</dia>  
22      <metge>Aura Pardo Valentín</metge>  
23      <pacient>  
24        <id_pacient>39532</id_pacient>  
25        <nom>Adalberto</nom>  
26        <cognom>Olivera</cognom>  
27        <cognom2>Abella</cognom2>  
28      </pacient>  
29    </visita>  
30  </visites>  
31 </exportacio_visites>
```

Validació amb JSON Schema i XSD

A part dels fitxers d'exportació, també s'han preparat els esquemes de validació dels dos formats.

Per a JSON s'ha creat un JSON Schema. Aquest fitxer defineix quins camps han d'existir, quins tipus de dades han de tenir i com s'organitza tota l'estructura del document.

Per a XML s'ha creat un fitxer XSD. Aquest esquema fa la mateixa funció que el JSON Schema, però aplicada al format XML. Serveix per comprovar que el document XML segueix exactament l'estructura prevista.

Gràcies a aquests esquemes, es pot validar que els fitxers exportats siguin correctes i consistents. Això és important perquè dona més fiabilitat a la funcionalitat i facilita el treball amb altres eines.

Els esquemes que s'han fet servir es poden trobar a [l'annex 4](#).

Dashboard PowerBI

Objectiu del Dashboard

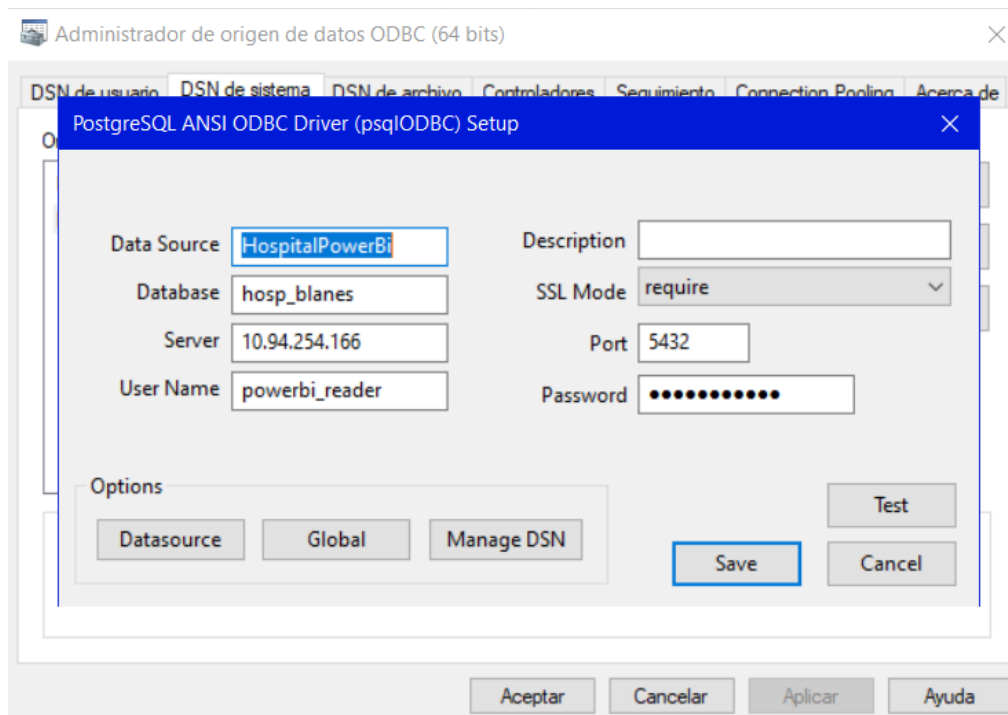
En aquesta part del projecte hem creat un quadre de comandament amb Power BI per mostrar de manera visual l'activitat de l'hospital. L'objectiu principal és que l'usuari pugui veure ràpidament la informació més important sense haver d'entrar a consultar taules o informes detallats.

El requisit mínim demanat era mostrar les visites totals d'avui i el seu desglossament per àrea. Per aquest motiu, el Dashboard s'ha plantejat perquè aquesta informació aparegui de manera clara a la pantalla principal.

Font de dades i connexió

Per construir el Dashboard, Power BI es connecta a la base de dades PostgreSQL del projecte. En lloc de treballar directament amb totes les taules, s'han preparat vistes específiques que ja deixen les dades agrupades i ordenades per facilitar la creació dels visuals.

Aquesta solució és útil perquè simplifica molt la feina dins de Power BI. A més, també s'ha preparat un usuari de només lectura per fer la connexió de manera més segura i separada de la resta de perfils de l'aplicació.

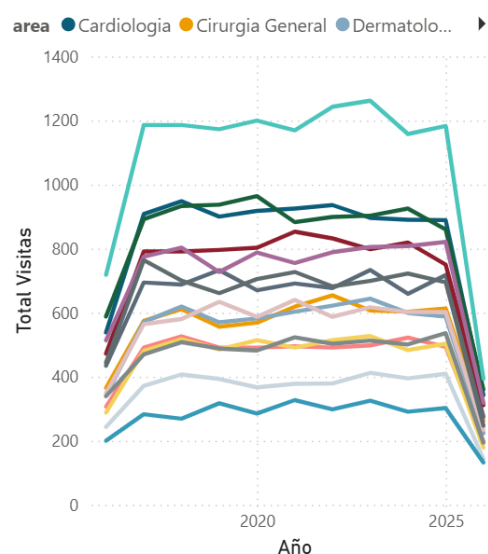


Desglossament de visites per àrea

El Dashboard mostra com es reparteixen aquestes visites per àrees o especialitats mèdiques, com per exemple traumatologia, cardiologia o altres serveis.

Aquest desglossament és útil perquè permet veure de manera visual quines àrees concentren més activitat. Així es pot interpretar millor el volum de feina i comparar fàcilment el pes de cada servei dins del conjunt de visites del dia.

Evolució de visites per àrea



Altres visuals de suport

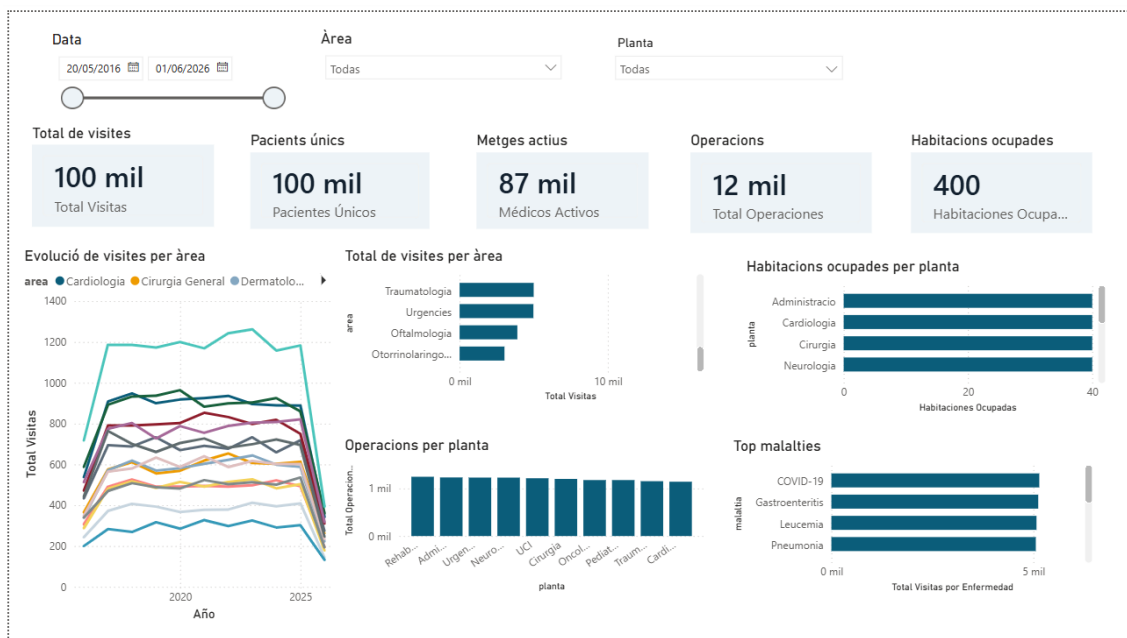
Encara que l'enunciat només demanava un mínim de dues dades, el Dashboard es pot complementar amb altres visuals de suport per donar una visió més completa de l'hospital. Per exemple, es poden mostrar visites per metge, franges horàries amb més activitat, ocupació d'habitacions, activitat de quiròfans o malalties més freqüents.

Aquests elements no són imprescindibles per complir el requisit mínim, però ajuden a convertir el Dashboard en una eina més útil i més propera a un entorn real.

Disseny i resultat final

A nivell visual, el Dashboard s'ha organitzat perquè la informació principal es vegi primer, amb targetes resum a la part superior i gràfics de suport a la resta de la pàgina. També s'ha aplicat un estil visual coherent amb la resta del projecte, de manera que el quadre de comandament manté una presentació clara i ordenada.

En conjunt, el resultat final compleix el que demanava l'enunciat: mostrar les visites totals d'avui i el seu desglossament per àrea mitjançant una eina tipus Power BI, amb una presentació visual fàcil d'entendre.



Enllaços d'interès

Enllaç a repositori GitHub: <https://github.com/lleerrriiiccc/Projecte-hospital>

Enllaç a carpeta de documentació: [DOC_DRIVE](#)

Annex 1

Model relacional

PERSONAL(

id_intern,

nom,

cognom1,

cognom2,

telèfon,

telefon2,

data_naixement,

email,

email_intern,

dni,

tipus_feina,

data_alta

)

METGE(

id_metge ON id_intern REFERENCIA (personal(id_intern)

especialitat,

cv

)

ENFERMER(

id_enfermer ON id_intern REFERENCIA (personal(id_intern)

)

PACIENTS(


```
id_pacient ,  
nom,  
cognom1,  
cognom2,  
identificacio,  
data_naixement,  
)
```

```
PLANTA(  
id_planta,  
nom  
)
```

```
HABITACIONS(  
numero_habitacio,  
id_planta ON id_planta referencia PLANTA(id_planta)  
)
```

```
VISITA(  
id_visit,  
data,  
hora,  
diagnostic,  
id_pacient  
id_metge  
ON id_pacient REFERENCIA pacient(id_pacient)  
ON id_metge REFERENCIA metge(id_metge)  
)
```

```
MEDICAMENTS(  
id_medicacio,
```

descripcio
)

RECEPTA(
 id_visita, id_medicacio,
 ON id_visita REFERENCIA visita(id_visita),
 ON id_medicacio REFERENCIA medicaments(id_medicacio)
)

OPERACIO(
 id_operacio,
 data,
 hora,
 procediment,
 id_pacient,
 id_metge,
 id_quirofan
 ON id_pacient REFERENCIA pacient(id_pacient)
 ON id_metge REFERENCIA metgre(id_intern)
 ON id_quirofan REFERENCIA quirofan(id_quirofan)
)

ASSISTEIX(
 id_enfermer,
 id_operacio,
 PRIMARY KEY(id_enfermer, id_operacio)
 ON id_enfermer REFERENCIA enfermer(id_intern)
 ON id_operacio REFERENCIA operacio(id_operacio)
)

```
QUIROFAN(  
    id_quirofan,  
    id_planta  
ON id_planta REFERENCIA planta(id_planta)  
)
```

```
MAQUINES(  
    id_model,  
    nom,  
    descripcio  
)
```

```
INVENTARI(  
    id_quirofan,  
    id_model,  
    PRIMARY KEY(id_quirofan, id_model)  
ON id_quirofan REFERENCIA quirofan(id_quirofan)  
ON id_model REFERENCIA maquina(id_model)  
)
```

```
RESERVA(  
    id_pacient,  
    numero_habitacio,  
    data_entrada,  
    data_sortida  
    PRIMARY KEY(id_pacient, numero_habitacio, data_entrada)  
ON id_pacient REFERENCIA pacient(id_pacient)  
ON numero_habitacio REFERENCIA habitacio(numero_habitacio)  
)
```

```
SUPERVISIO(  

```

```

    id_intern,
    id_metge
ON id_intern REFERENCIA personal(id_intern)
ON id_metge REFERENCIA metge(id_intern)
)

```

```

USUARIS(
    id_user,
    username,
    password,
    id_intern,
    rol
ON id_intern REFERENCIA personal(id_intern)
)

```

Script SQL implementació

```

--PERSONAL
CREATE TABLE personal (
    id_intern SERIAL PRIMARY KEY,
    nom VARCHAR(50) NOT NULL,
    cognom VARCHAR(50) NOT NULL,
    cognom2 VARCHAR(50) NOT NULL,
    data_naixement DATE NOT NULL,
    telefon VARCHAR(15) NOT NULL,
    telefon2 VARCHAR(15) NOT NULL,
    email VARCHAR(100) NOT NULL CHECK (email LIKE '%@%'),
    email_intern VARCHAR(100) NOT NULL CHECK (email_intern LIKE '%@%'),
    dni VARCHAR(20) NOT NULL UNIQUE,
    tipus_feina VARCHAR(50) NOT NULL,
    data_alta DATE NOT NULL
)

```

```
);
```

```
--METGE
```

```
CREATE TABLE metge (
```

```
    id_intern INT NOT NULL PRIMARY KEY,
```

```
    especialitat VARCHAR(50) NOT NULL,
```

```
    cv VARCHAR(255) NOT NULL,
```

```
    FOREIGN KEY (id_intern) REFERENCES personal(id_intern)
```

```
);
```

```
--ENFERMER
```

```
CREATE TABLE enfermer (
```

```
    id_intern INT NOT NULL PRIMARY KEY,
```

```
    FOREIGN KEY (id_intern) REFERENCES personal(id_intern)
```

```
);
```

```
--PACIENT
```

```
CREATE TABLE pacient (
```

```
    id_pacient SERIAL PRIMARY KEY,
```

```
    nom VARCHAR(50) NOT NULL,
```

```
    cognom VARCHAR(50) NOT NULL,
```

```
    cognom2 VARCHAR(50) NOT NULL,
```

```
    data_naixement DATE NOT NULL,
```

```
    identificador VARCHAR(20) NOT NULL UNIQUE
```

```
);
```

```
--PLANTA
```

```
CREATE TABLE planta (
```

```
    id_planta SERIAL PRIMARY KEY,
```

```
    nom VARCHAR(50) NOT NULL
```

```
);
```

--HABITACIO

```
CREATE TABLE habitacio (  
    id_planta INT NOT NULL,  
    num_habitacio VARCHAR(10) NOT NULL PRIMARY KEY,  
    FOREIGN KEY (id_planta) REFERENCES planta(id_planta)  
);
```

--MEDICAMENT

```
CREATE TABLE medicament (  
    id_medicament SERIAL PRIMARY KEY,  
    descripcio TEXT NOT NULL  
);
```

--MALALTIA

```
CREATE TABLE malaltia (  
    id_malaltia SERIAL PRIMARY KEY,  
    nom VARCHAR(100) NOT NULL UNIQUE  
);
```

--QUIROFAN

```
CREATE TABLE quirofan (  
    id_quirofan SERIAL PRIMARY KEY,  
    id_planta INT NOT NULL,  
    FOREIGN KEY (id_planta) REFERENCES planta(id_planta)  
);
```

--MAQUINES

```
CREATE TABLE maquina (  
    id_maquina SERIAL PRIMARY KEY,  
    nom VARCHAR(50) NOT NULL,
```

```
descripcio TEXT NOT NULL  
);
```

```
--VISITA
```

```
CREATE TABLE visita (  
    id_visita SERIAL PRIMARY KEY,  
    id_pacient INT NOT NULL,  
    id_metge INT NOT NULL,  
    id_malaltia INT,  
    data_visita DATE NOT NULL,  
    hora_visita TIME NOT NULL,  
    diagnostic TEXT NOT NULL,  
    FOREIGN KEY (id_pacient) REFERENCES pacient(id_pacient),  
    FOREIGN KEY (id_metge) REFERENCES metge(id_intern),  
    FOREIGN KEY (id_malaltia) REFERENCES malaltia(id_malaltia)  
);
```

```
--RECEPTA
```

```
CREATE TABLE recepta (  
    id_visita INT NOT NULL,  
    id_medicament INT NOT NULL,  
    FOREIGN KEY (id_visita) REFERENCES visita(id_visita),  
    FOREIGN KEY (id_medicament) REFERENCES medicament(id_medicament),  
    PRIMARY KEY (id_visita, id_medicament)  
);
```

```
--OPERACIO
```

```
CREATE TABLE operacio (  
    id_operacio SERIAL PRIMARY KEY,  
    id_quirofan INT NOT NULL,  
    id_pacient INT NOT NULL,
```

```

data_operacio DATE NOT NULL,
hora_operacio TIME NOT NULL,
procediment TEXT NOT NULL,
metge_responsable INT NOT NULL,
FOREIGN KEY (id_quirofan) REFERENCES quirofan(id_quirofan),
FOREIGN KEY (id_pacient) REFERENCES pacient(id_pacient),
FOREIGN KEY (metge_responsable) REFERENCES metge(id_intern)
);

```

--ASSISTEIX

```

CREATE TABLE assisteix (
    id_operacio INT NOT NULL,
    id_intern INT NOT NULL,
    FOREIGN KEY (id_operacio) REFERENCES operacio(id_operacio),
    FOREIGN KEY (id_intern) REFERENCES personal(id_intern),
    PRIMARY KEY (id_operacio, id_intern)
);

```

--Inventari

```

CREATE TABLE inventari (
    id_quirofan INT NOT NULL,
    id_maquina INT NOT NULL,
    FOREIGN KEY (id_maquina) REFERENCES maquina(id_maquina),
    FOREIGN KEY (id_quirofan) REFERENCES quirofan(id_quirofan),
    PRIMARY KEY (id_quirofan, id_maquina)
);

```

--RESERVA_HABITACIO

```

CREATE TABLE reserva_habitacio (
    id_pacient INT NOT NULL,
    num_habitacio VARCHAR(10) NOT NULL,

```



```

    data_inici DATE NOT NULL,
    data_fi DATE NOT NULL,
    FOREIGN KEY (id_pacient) REFERENCES pacient(id_pacient),
    FOREIGN KEY (num_habitacio) REFERENCES habitacio(num_habitacio),
    PRIMARY KEY (id_pacient, num_habitacio, data_inici)
);

```

--SUPERVISIO

```

CREATE TABLE supervisio (
    id_intern INT NOT NULL,
    id_metge INT NOT NULL,
    FOREIGN KEY (id_intern) REFERENCES personal(id_intern),
    FOREIGN KEY (id_metge) REFERENCES metge(id_intern),
    PRIMARY KEY (id_intern, id_metge)
);

```

--ASSIGNACIO_INFERMER_PLANTA

```

CREATE TABLE assignacio_infermer_planta (
    id_intern INT NOT NULL,
    id_planta INT NOT NULL,
    FOREIGN KEY (id_intern) REFERENCES enfermer(id_intern),
    FOREIGN KEY (id_planta) REFERENCES planta(id_planta),
    PRIMARY KEY (id_intern, id_planta)
);

```

--USUARIS

```

CREATE TABLE usuaris (
    id_user SERIAL PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    id_intern INT NOT NULL,

```

```

rol VARCHAR(20) NOT NULL,

FOREIGN KEY (id_intern) REFERENCES personal(id_intern)

);

CREATE INDEX personal_tipus_feina ON personal(tipus_feina);

CREATE INDEX visita_data_visita ON visita(data_visita);
CREATE INDEX visita_id_malaltia ON visita(id_malaltia);

-- MIGRACIO DES DE LA VERSIO ACTUAL

-- Executa aquest bloc si ja tens la base de dades creada amb L'esquema
anterior

-- i vols afegir la relacio entre infermers i plantes sense reconstruir-ho
tot.

DO $$
BEGIN
    IF NOT EXISTS (
        SELECT 1
        FROM information_schema.tables
        WHERE table_schema = 'public'
        AND table_name = 'assignacio_infermer_planta'
    ) THEN
        CREATE TABLE assignacio_infermer_planta (
            id_intern INT NOT NULL,
            id_planta INT NOT NULL,
            FOREIGN KEY (id_intern) REFERENCES enfermer(id_intern),
            FOREIGN KEY (id_planta) REFERENCES planta(id_planta),
            PRIMARY KEY (id_intern, id_planta)
        );
    END IF;
END $$;

```

```

-- MIGRACIO PER A LES MALALTIES

-- Executa aquest bloc si ja tens l'esquema anterior i vols convertir els
-- diagnòstics de les visites en una taula normalitzada de malalties.

DO $$
BEGIN
    IF NOT EXISTS (
        SELECT 1
        FROM information_schema.tables
        WHERE table_schema = 'public'
        AND table_name = 'malaltia'
    ) THEN
        CREATE TABLE malaltia (
            id_malaltia SERIAL PRIMARY KEY,
            nom VARCHAR(100) NOT NULL UNIQUE
        );
    END IF;

    IF NOT EXISTS (
        SELECT 1
        FROM information_schema.columns
        WHERE table_schema = 'public'
        AND table_name = 'visita'
        AND column_name = 'id_malaltia'
    ) THEN
        ALTER TABLE visita ADD COLUMN id_malaltia INT;
    END IF;
END $$;

INSERT INTO malaltia (nom)
VALUES

```

```

('Grip'),
('Bronquitis'),
('Migranya')
ON CONFLICT (nom) DO NOTHING;

INSERT INTO malaltia (nom)
SELECT DISTINCT INITCAP(TRIM(diagnostic))
FROM visita
WHERE diagnostic IS NOT NULL AND TRIM(diagnostic) <> ''
ON CONFLICT (nom) DO NOTHING;

UPDATE visita v
SET id_malaltia = m.id_malaltia
FROM malaltia m
WHERE LOWER(TRIM(v.diagnostic)) = LOWER(TRIM(m.nom));

DO $$
BEGIN
    IF NOT EXISTS (
        SELECT 1
        FROM information_schema.table_constraints
        WHERE table_schema = 'public'
        AND table_name = 'visita'
        AND constraint_name = 'visita_id_malaltia_fkey'
    ) THEN
        ALTER TABLE visita
        ADD CONSTRAINT visita_id_malaltia_fkey
        FOREIGN KEY (id_malaltia) REFERENCES malaltia(id_malaltia);
    END IF;
END $$;

```

Annex2

Esquema de seguretat

```
-- Matriu de seguretat

-- - Es mantenen dos usuaris tècnics de PostgreSQL.

-- - Es creen rols de login per a cada tipus de persona i rols de grup per agrupar permisos.

-- - Els rols de login es creen amb el prefix rol_ per no xocar amb els rols de grup.

-- - Els usuaris reals de l'aplicació es guarden a la taula usuaris.

-- - El backend en Python continua usant un usuari tècnic propi.


-- Rols de PostgreSQL que es creen:

-- 1) hosp_admin: administra tota la base de dades.

-- 2) hosp_app: usuari tècnic que farà servir l'aplicació web.

-- 3) administrador, personal, sanitari, gestio, serveis i pacient: rols de grup.

-- 4) rol_administrador, rol_metge, rol_infermer, rol_administratiu, rol_tecnic,

--     rol_personal_neteja, rol_personal_seguretat, rol_personal_cuina i rol_pacient:

--     rols de login amb contrasenya d'exemple P@ssw0rd.


BEGIN;


REVOKE CONNECT ON DATABASE hosp_blanes FROM PUBLIC;

REVOKE ALL ON SCHEMA public FROM PUBLIC;

REVOKE ALL ON ALL TABLES IN SCHEMA public FROM PUBLIC;

REVOKE ALL ON ALL SEQUENCES IN SCHEMA public FROM PUBLIC;


DO $$

BEGIN

    IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'hosp_admin') THEN
```

```

CREATE ROLE hosp_admin LOGIN CREATEDB CREATEROLE PASSWORD
'P@ssw0rd!';

END IF;

```

```

IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'hosp_app') THEN

CREATE ROLE hosp_app LOGIN PASSWORD 'P@ssw0rd!app';

GRANT SUPERUSER TO hosp_app;

END IF;

```

```

IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'administrador')
THEN

CREATE ROLE administrador NOLOGIN;

END IF;

```

```

IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'personal') THEN

CREATE ROLE personal NOLOGIN;

END IF;

```

```

IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'sanitari') THEN

CREATE ROLE sanitari NOLOGIN;

END IF;

```

```

IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'gestio') THEN

CREATE ROLE gestio NOLOGIN;

END IF;

```

```

IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'serveis') THEN

CREATE ROLE serveis NOLOGIN;

END IF;

```

```

IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'pacient') THEN

```

```

CREATE ROLE pacient NOLOGIN;

END IF;

IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname =
'rol_administrador') THEN

CREATE ROLE rol_administrador LOGIN PASSWORD 'P@ssw0rd';

ELSE

ALTER ROLE rol_administrador LOGIN PASSWORD 'P@ssw0rd';

END IF;

IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'rol_metge') THEN

CREATE ROLE rol_metge LOGIN PASSWORD 'P@ssw0rd';

ELSE

ALTER ROLE rol_metge LOGIN PASSWORD 'P@ssw0rd';

END IF;

IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'rol_infermer')
THEN

CREATE ROLE rol_infermer LOGIN PASSWORD 'P@ssw0rd';

ELSE

ALTER ROLE rol_infermer LOGIN PASSWORD 'P@ssw0rd';

END IF;

IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname =
'rol_administratiu') THEN

CREATE ROLE rol_administratiu LOGIN PASSWORD 'P@ssw0rd';

ELSE

ALTER ROLE rol_administratiu LOGIN PASSWORD 'P@ssw0rd';

END IF;

IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'rol_tecnic') THEN

CREATE ROLE rol_tecnic LOGIN PASSWORD 'P@ssw0rd';

```

```

ELSE
    ALTER ROLE rol_tecnic LOGIN PASSWORD 'P@ssw0rd';
END IF;

IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname =
'rol_personal_neteja') THEN
    CREATE ROLE rol_personal_neteja LOGIN PASSWORD 'P@ssw0rd';
ELSE
    ALTER ROLE rol_personal_neteja LOGIN PASSWORD 'P@ssw0rd';
END IF;

IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname =
'rol_personal_seguretat') THEN
    CREATE ROLE rol_personal_seguretat LOGIN PASSWORD 'P@ssw0rd';
ELSE
    ALTER ROLE rol_personal_seguretat LOGIN PASSWORD 'P@ssw0rd';
END IF;

IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname =
'rol_personal_cuina') THEN
    CREATE ROLE rol_personal_cuina LOGIN PASSWORD 'P@ssw0rd';
ELSE
    ALTER ROLE rol_personal_cuina LOGIN PASSWORD 'P@ssw0rd';
END IF;

IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'rol_pacient') THEN
    CREATE ROLE rol_pacient LOGIN PASSWORD 'P@ssw0rd';
ELSE
    ALTER ROLE rol_pacient LOGIN PASSWORD 'P@ssw0rd';
END IF;
END
$$;

```



```

GRANT CONNECT ON DATABASE hosp_blanes TO hosp_admin, hosp_app,
    rol_administrador, rol_metge, rol_infermer, rol_administratiu,
    rol_tecnic,
    rol_personal_neteja, rol_personal_seguretat, rol_personal_cuina,
    rol_pacient;

GRANT USAGE, CREATE ON SCHEMA public TO hosp_admin;

GRANT USAGE ON SCHEMA public TO hosp_app,
    rol_administrador, rol_metge, rol_infermer, rol_administratiu,
    rol_tecnic,
    rol_personal_neteja, rol_personal_seguretat, rol_personal_cuina,
    rol_pacient;

-- Administració tècnica total.

GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO hosp_admin;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO hosp_admin;

-- Perfil funcional d'administrador.

GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO administrador;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO administrador;

-- Perfil funcional de personal.

GRANT SELECT ON TABLE
    personal,
    planta,
    habitacio,
    medicament,
    malaltia,
    quirofan,
    maquina,
    assignacio_infermer_planta
TO personal;

```

```
-- Perfil funcional sanitari.
```

```
GRANT SELECT, INSERT, UPDATE ON TABLE
```

```
personal,
```

```
metge,
```

```
enfermer,
```

```
pacient,
```

```
visita,
```

```
malaltia,
```

```
recepta,
```

```
operacio,
```

```
assisteix,
```

```
supervisio
```

```
TO sanitari;
```

```
GRANT SELECT ON TABLE
```

```
planta,
```

```
habitacio,
```

```
medicament,
```

```
malaltia,
```

```
quirofan,
```

```
maquina,
```

```
inventari,
```

```
reserva_habitacio,
```

```
assignacio_infermer_planta
```

```
TO sanitari;
```

```
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO sanitari;
```

```
-- Perfil funcional de gestió.
```

```
GRANT SELECT, INSERT, UPDATE ON TABLE
```

```

personal,
metge,
enfermer,
pacient,
planta,
habitacio,
medicament,
malaltia,
quirofan,
maquina,
visita,
inventari,
reserva_habitacio,
assignacio_infermer_planta
TO gestio;

GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO gestio;

-- Perfil funcional de serveis.
GRANT SELECT ON TABLE
planta,
habitacio,
malaltia,
quirofan,
maquina,
inventari
TO serveis;

-- Permisos del backend.
-- El backend necessita llegir i modificar dades, però no fer DDL.
GRANT SELECT, INSERT, UPDATE ON TABLE

```

```
personal,  
metge,  
enfermer,  
pacient,  
planta,  
habitacio,  
medicament,  
malaltia,  
quiroman,  
maquina,  
visita,  
recepta,  
operacio,  
assisteix,  
inventari,  
reserva_habitacio,  
supervisio,  
assignacio_infermer_planta,  
usuari  
TO hosp_app;
```

```
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO hosp_app;
```

```
ALTER DEFAULT PRIVILEGES FOR ROLE hosp_admin IN SCHEMA public
```

```
GRANT SELECT, INSERT, UPDATE ON TABLES TO hosp_app;
```

```
ALTER DEFAULT PRIVILEGES FOR ROLE hosp_admin IN SCHEMA public
```

```
GRANT USAGE, SELECT ON SEQUENCES TO hosp_app;
```

```
ALTER DEFAULT PRIVILEGES FOR ROLE hosp_admin IN SCHEMA public
```

```
GRANT ALL PRIVILEGES ON TABLES TO administrador;
```

```
ALTER DEFAULT PRIVILEGES FOR ROLE hosp_admin IN SCHEMA public
GRANT ALL PRIVILEGES ON SEQUENCES TO administrador;
```

```
ALTER DEFAULT PRIVILEGES FOR ROLE hosp_admin IN SCHEMA public
GRANT SELECT ON TABLES TO personal;
```

```
ALTER DEFAULT PRIVILEGES FOR ROLE hosp_admin IN SCHEMA public
GRANT SELECT, INSERT, UPDATE ON TABLES TO sanitari;
```

```
ALTER DEFAULT PRIVILEGES FOR ROLE hosp_admin IN SCHEMA public
GRANT USAGE, SELECT ON SEQUENCES TO sanitari;
```

```
ALTER DEFAULT PRIVILEGES FOR ROLE hosp_admin IN SCHEMA public
GRANT SELECT, INSERT, UPDATE ON TABLES TO gestio;
```

```
ALTER DEFAULT PRIVILEGES FOR ROLE hosp_admin IN SCHEMA public
GRANT USAGE, SELECT ON SEQUENCES TO gestio;
```

```
ALTER DEFAULT PRIVILEGES FOR ROLE hosp_admin IN SCHEMA public
GRANT SELECT ON TABLES TO serveis;
```

-- Matriu funcional de l'aplicació:

-- administrador: accés total dins de l'aplicació.

-- personal: accés base de lectura a dades comunes.

-- sanitari: metges i infermers.

-- gestio: administratius i tècnics.

-- serveis: personal de neteja, seguretat i cuina.

-- pacient: rol preparat per a un accés restringit amb vistes o RLS.

-- Els rols funcionals existeixen a PostgreSQL i els rols de login s'assignen als grups corresponents.

```
GRANT administrador TO rol_administrador;  
  
GRANT personal TO rol_metge, rol_infermer, rol_administratiu, rol_tecnic,  
    rol_personal_neteja, rol_personal_seguretat, rol_personal_cuina;  
  
GRANT sanitari TO rol_metge, rol_infermer;  
  
GRANT gestio TO rol_administratiu, rol_tecnic;  
  
GRANT serveis TO rol_personal_neteja, rol_personal_seguretat,  
    rol_personal_cuina;  
  
GRANT pacient TO rol_pacient;
```

```
ALTER ROLE rol_administrador OWNER TO hosp_app;  
  
ALTER ROLE rol_metge OWNER TO hosp_app;  
  
ALTER ROLE rol_infermer OWNER TO hosp_app;  
  
ALTER ROLE rol_administratiu OWNER TO hosp_app;  
  
ALTER ROLE rol_tecnic OWNER TO hosp_app;  
  
ALTER ROLE rol_personal_neteja OWNER TO hosp_app;  
  
ALTER ROLE rol_personal_seguretat OWNER TO hosp_app;  
  
ALTER ROLE rol_personal_cuina OWNER TO hosp_app;  
  
ALTER ROLE rol_pacient OWNER TO hosp_app;
```

```
COMMIT;
```

Annex 3

Instal·lació tailscale

Motiu d'us

La nostra infraestructura de replicació requereix una connexió entre cloud i xarxa privada degut a que estem sota NAT del ISP i protegits pel Firewall. Degut a aquesta problemàtica hem investigat solucions. Entre aquestes les següents.

- Túnel TCP → Ngrok
- Túnel SSH → ssh genèric
- VPN → Tailscale

Degut a que els túnels tcp amb Ngrok es probable que sigui necessari el pagament i que son limitats hem decidit no utilitzar aquesta opció. La següent opció eren els túnels SSH, degut a la possible inestabilitat hem decidit no utilitzar aquesta tecnologia.

La tercera opció es una xarxa virtual amb túnels TCP utilitzant Tailscale, aquesta opció es la mes estable encara que la mes complicada d'implementar. Es opensource per tant gratuïta, l'únic inconvenient es la possibilitat que no funcioni correctament dins del nostre proveïdor de cloud, si aquest es el cas es decidiria quin dels dos altres mètodes farem servir enlloc de la VPN.

Servidor Local

En aquest apartat veurem la instal·lació i configuració en el servidor local per instal·lar Tailscale.

Instal·lació

Executar la següent comanda per descarregar i instal·lar Tailscale

```
curl -fsSL https://tailscale.com/install.sh | sh
```

```
eric@serverhosp:~$ curl -fsSL https://tailscale.com/install.sh | sh
```

```
Reading package lists... Done
+ [ -n ]
+ sudo apt-get install -y tailscale tailscale-archive-keyring
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  tailscale tailscale-archive-keyring
0 upgraded, 2 newly installed, 0 to remove and 35 not upgraded.
Need to get 36.6 MB of archives.
After this operation, 71.9 MB of additional disk space will be used.
Get:1 https://pkgs.tailscale.com/stable/ubuntu noble/main amd64 tailscale amd64 1.96.4 [36.6 MB]
55% [1 tailscale 25.2 MB/36.6 MB 69%] 2,386 kB/s 4s
```

Installation complete! Log in to start using Tailscale by running:

```
sudo tailscale up
eric@serverhosp:~$ |
```

Després de la execució d'aquesta comanda ja queda instal·lat el servei.

Primeres passes

Ara toca aixecar el servei i veure si dona algun error

Per fer això executem la següent comanda

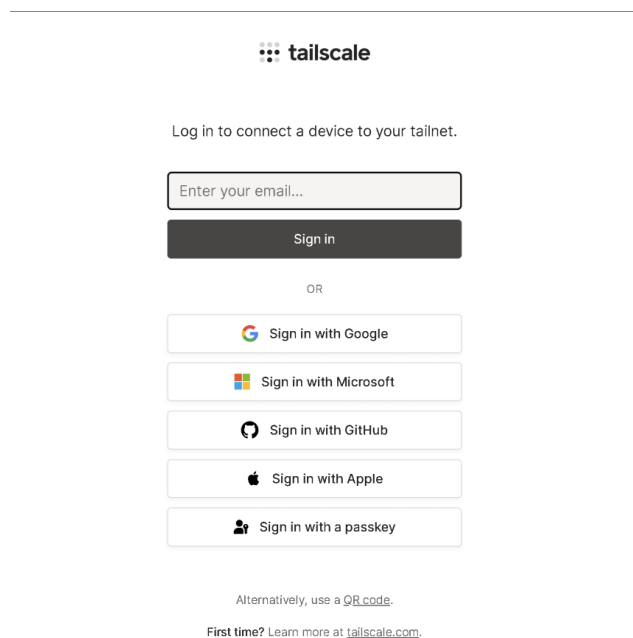
sudo tailscale up

```
eric@serverhosp:~$ sudo tailscale up

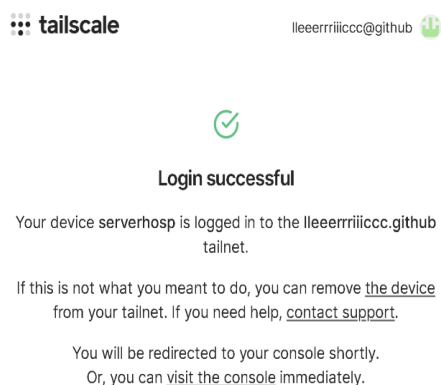
To authenticate, visit:

https://login.tailscale.com/a/1a572dc0017e8b
```

Després de la execució ens mostrarà aquesta URL, hem d'accedir per configurar la nostra compte.



Podem accedir de diferents maneres, jo accediré amb el meu perfil de GitHub.



Després d'iniciar sessió veurem aquesta pantalla.

Verificacions

Ara tocarà verificar si esta funcionant i quina ip ens ha donat a la maquina.

Per fer-ho utilitzarem les següents comandes.

Tailscale status

```
eric@serverhosp:~$ tailscale status
100.105.147.27 serverhosp lleeerrriiiccc@ linux -
eric@serverhosp:~$ |
```

Tailscale ip

```
eric@serverhosp:~$ tailscale ip
100.105.147.27
fd7a:115c:a1e0::5138:931b
eric@serverhosp:~$ |
```

Un cop ja tenim la informació començarem amb la post instal·lació

Post instal·lació

Ara configurarem el servei perquè s'activi al iniciar el servidor

sudo systemctl enable tailscaled

```
eric@serverhosp:~$ sudo systemctl enable tailscaled
eric@serverhosp:~$ |
```

Servidor Cloud

/Instal·lació

Per instal·lar tailscale al servidor cloud remot executarem la mateixa comanda que al servidor local.

curl -fsSL https://tailscale.com/install.sh | sh

```
root@aiproject-vps:~# curl -fsSL https://tailscale.com/install.sh | sh
Installing Tailscale for ubuntu noble, using method apt
+ mkdir -p --mode=0755 /usr/share/keyrings
+ curl -fsSL https://pkgs.tailscale.com/stable/ubuntu/noble.noarmor.gpg
```

```
no in guests are running nested hypervisor (qemu) binaries on this m
+ [ false = true ]
+ set +x
Installation complete! Log in to start using Tailscale by running:
tailscale up
```

Primeres pases

Primer iniciarem el servei i després iniciarem sessió per afegir el dispositiu a la meva compta.

Executarem la següent comanda

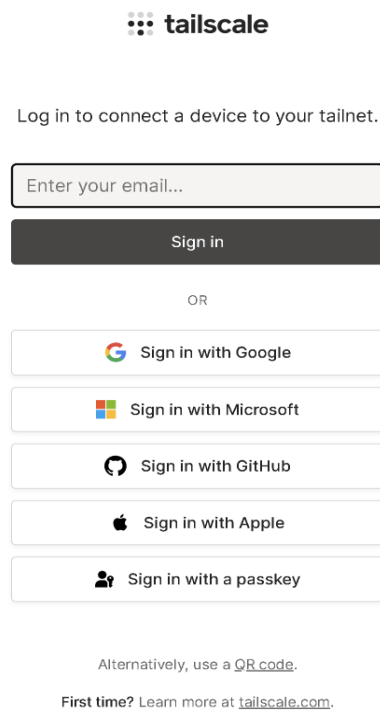
sudo tailscale up

```
root@aiproject-vps:~# sudo tailscale up

To authenticate, visit:

https://login.tailscale.com/a/10d17ba701036a
```

Accedim al link i veurem la següent pantalla



De la mateixa manera que amb el local ens demana iniciar sessió, jo faré servir GitHub

**Login successful**

Your device `aiproject-vps` is logged in to the
`lleeerrriiiccc.github` tailnet.

If this is not what you meant to do, you can remove [the device](#)
 from your tailnet. If you need help, [contact support](#).

You will be redirected to your console shortly.
 Or, you can [visit the console](#) immediately.

Un cop afegir el nostre dispositiu veurem aquesta pagina.

Verificacions

Ara verificarem que el servei estigui corrent i que estigui connectat a la nostra xarxa.

```
root@aiproject-vps:~# tailscale status
100.101.108.31  aiproject-vps  lleeerrriiiccc@  linux  -
100.105.147.27  serverhosp      lleeerrriiiccc@  linux  -
root@aiproject-vps:~# |
```

Podem veure els nostres dos dispositius en aquesta xarxa.

Ara provarem de fer ping d'un al altre.

Des de cloud a local.

```
root@aiproject-vps:~# ping serverhosp -c 1
PING serverhosp.tail68645b.ts.net (100.105.147.27) 56(84) bytes of data.
64 bytes from serverhosp.tail68645b.ts.net (100.105.147.27): icmp_seq=1 ttl=64 time=58.3 ms

--- serverhosp.tail68645b.ts.net ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 58.261/58.261/58.261/0.000 ms
root@aiproject-vps:~# |
```

Des de local a cloud

```
eric@serverhosp:~$ ping aiproject-vps -c 1
PING aiproject-vps.tail68645b.ts.net (100.101.108.31) 56(84) bytes of data.
64 bytes from aiproject-vps.tail68645b.ts.net (100.101.108.31): icmp_seq=1 ttl=64 time=75.2 ms

--- aiproject-vps.tail68645b.ts.net ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 75.245/75.245/75.245/0.000 ms
eric@serverhosp:~$ |
```

Post instal·lació

Ara habilitarem que el servei s'aisequi durant la arrancada del servidor.

```
root@aiproject-vps:~# systemctl enable tailscaled
root@aiproject-vps:~# |
```

Annex 4

Esquema json

Aquí posem la estructura dels dos schemas:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "visites_export.schema.json",
  "title": "Exportació de visites",
  "type": "object",
  "additionalProperties": false,
  "required": ["data_inici", "data_fi", "visites"],
  "properties": {
    "data_inici": {
      "type": "string",
      "format": "date"
    },
    "data_fi": {
      "type": "string",
      "format": "date"
    },
    "visites": {
      "type": "array",
      "items": {
        "type": "object",
        "additionalProperties": false,
        "required": ["id_visita", "dia", "metge", "pacient"],
        "properties": {
          "id_visita": {
            "type": "integer",
```

```
    "minimum": 1
  },
  "dia": {
    "type": "string",
    "format": "date"
  },
  "metge": {
    "type": "string",
    "minLength": 1
  },
  "pacient": {
    "type": "object",
    "additionalProperties": false,
    "required": ["id_pacient", "nom", "cognom", "cognom2"],
    "properties": {
      "id_pacient": {
        "type": "integer",
        "minimum": 1
      },
      "nom": {
        "type": "string",
        "minLength": 1
      },
      "cognom": {
        "type": "string",
        "minLength": 1
      },
      "cognom2": {
        "type": "string",
        "minLength": 1
      }
    }
  }
}
```

$$\left\{ \begin{array}{l} \\ \\ \\ \\ \\ \end{array} \right\}$$

Esquema XSD

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="exportacio_visites">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="rang_dates">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="data_inici" type="xs:date"/>
              <xs:element name="data_fi" type="xs:date"/>
            </xs:sequence>
          </xs:complexType>
        </xs:element>
        <xs:element name="visites">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="visita" minOccurs="0" maxOccurs="unbounded">
                <xs:complexType>
                  <xs:sequence>
                    <xs:element name="id_visita" type="xs:positiveInteger"/>
                    <xs:element name="dia" type="xs:date"/>
                    <xs:element name="metge" type="xs:string"/>

```

```
<xs:element name="pacient">  
  <xs:complexType>  
    <xs:sequence>  
      <xs:element name="id_pacient" type="xs:positiveInteger"/>  
      <xs:element name="nom" type="xs:string"/>  
      <xs:element name="cognom" type="xs:string"/>  
      <xs:element name="cognom2" type="xs:string"/>  
    </xs:sequence>  
  </xs:complexType>  
</xs:element>  
</xs:sequence>  
</xs:complexType>  
</xs:element>  
</xs:sequence>  
</xs:complexType>  
</xs:element>  
</xs:sequence>  
</xs:complexType>  
</xs:element>  
</xs:sequence>  
</xs:complexType>  
</xs:element>  
</xs:schema>
```